

# Synopsis: A Scripting Language for Sufficient Alignment in Genomics

## Abstract

Genomic comparison is conducted almost entirely in the vocabulary of *exact* correspondence: percent identity, edit distance, alignment score. This vocabulary is not merely imprecise for the question practitioners actually ask — *do these two things do the same thing?* — it is provably unable to answer it. We prove two separation results. There exist pairs of units whose content is separated at the theoretical minimum, so that no content-based criterion can distinguish them, yet whose effect on the surrounding network diverges without bound; and there exist pairs whose content is arbitrarily expensive to reconcile yet whose effect is identical. Content comparison collapses the first class to *same* and the second to *different*, and is wrong in both directions. The two classes are orthogonal axes of a plane that a scalar criterion projects onto a line.

We construct the theory needed to occupy that plane. Working in finite weighted graphs with strictly positive weights, we establish a *contact floor*: a strictly positive lower bound  $\beta$  on the cost of any distinction. We show that identity is borne by a minimum cut — a region of edges — and never by a vertex or a label, so that it survives arbitrary relabelling exactly and is not carried by any name a database can store. We show that the value a unit registers is a function of the pair (unit, receiver) and not of the unit, and that the resulting disagreement between receivers is not measurement error. We then give a bidirectional comparison, *cross-demand*, that certifies correspondence without reading meaning, and prove that its relaxation terminates in at most  $\lceil D_0/\eta \rceil$  steps or declines — there is no third outcome and no divergence.

From this theory we design and specify **synopsis** (extension `.syp`), a total scripting language for genomic experiments. Its name is taken from the synoptic gospels, the books that are read in parallel columns because they *align sufficiently*, not because they agree word for word; the discipline of reading them is exactly the discipline we formalise. Synopsis is designed around refusals. Sequences are opaque and cannot be indexed. Every value carries a residue bounded below by the floor, and no constructor produces zero residue, so **identical** is not expressible. The verdict constructor is *4-ary*: omitting the response columns is an arity error, not a weaker result. Comparison across receiver frames is a scope error. Thresholds have no defaults. Iteration is bounded by construction and relaxation typechecks only when its step floor is bounded below, in which case the compiler emits the termination bound as a proof obligation it has discharged. We prove strong normalisation, floor preservation, scope safety, and four *refusal theorems* stating that certain scientifically invalid claims have no well-typed representation.

We give the full grammar, a typing judgement, a small-step operational semantics, and worked programs reproducing three families of genomic experiment — matched-filter motif detection, spectral homology search over a database, and four-column functional adjudication — in a single language whose output is a report bearing certificates rather than a number. We report honestly that one assumption the four-column verdict requires, the independence of the verdict from the choice of admissible response, *fails* on solved biochemical networks: only 28.6% of comparisons fall within threshold, with a maximum discrepancy of 0.307 against  $\theta = 0.01$ . Synopsis makes this failure syntactically visible: the response operator requires its map to be named, so no program can conceal which arbitrary choice it made. We regard designing a language that cannot hide its open problems as the point.

# Contents

|          |                                                             |           |
|----------|-------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                         | <b>4</b>  |
| 1.1      | Two failures of the same kind . . . . .                     | 4         |
| 1.2      | Sufficient alignment . . . . .                              | 4         |
| 1.3      | Where the invariant is not . . . . .                        | 5         |
| 1.4      | Why a language, and why a total one . . . . .               | 5         |
| 1.5      | Contributions and organisation . . . . .                    | 6         |
| <b>2</b> | <b>Contact structures and the floor</b>                     | <b>6</b>  |
| 2.1      | Finite weighted graphs . . . . .                            | 7         |
| 2.2      | The floor . . . . .                                         | 7         |
| 2.3      | Invariance under relabelling . . . . .                      | 8         |
| 2.4      | Expansion, and a false theorem made true . . . . .          | 8         |
| 2.5      | Perturbation . . . . .                                      | 9         |
| <b>3</b> | <b>Identity, region, and the receiver</b>                   | <b>9</b>  |
| 3.1      | Identity is a region, not a point . . . . .                 | 9         |
| 3.2      | Receiver-relativity . . . . .                               | 10        |
| <b>4</b> | <b>Blind comparison: demand, relaxation, quiescence</b>     | <b>11</b> |
| 4.1      | Alignment and distance to the floor . . . . .               | 11        |
| 4.2      | Anchors and residue . . . . .                               | 11        |
| 4.3      | Cross-demand . . . . .                                      | 12        |
| 4.4      | Relaxation . . . . .                                        | 12        |
| <b>5</b> | <b>The separation results and the four-column criterion</b> | <b>13</b> |
| 5.1      | Content-aligned, response-divergent . . . . .               | 13        |
| 5.2      | Content-divergent, response-aligned . . . . .               | 14        |
| 5.3      | The plane and the criterion . . . . .                       | 15        |
| 5.4      | The assumption that fails . . . . .                         | 15        |
| <b>6</b> | <b>Design: eleven rules</b>                                 | <b>16</b> |
| 6.1      | On Theorem 6.1, at length . . . . .                         | 17        |
| 6.2      | Three families, three index sets . . . . .                  | 17        |
| <b>7</b> | <b>The synopsis language</b>                                | <b>18</b> |
| 7.1      | Lexical structure . . . . .                                 | 18        |
| 7.2      | Types . . . . .                                             | 18        |
| 7.3      | Grammar . . . . .                                           | 18        |
| 7.4      | Frames and the <b>under</b> block . . . . .                 | 19        |
| 7.5      | Units and residue . . . . .                                 | 20        |
| 7.6      | Relaxation . . . . .                                        | 20        |
| 7.7      | Partition depth . . . . .                                   | 20        |
| 7.8      | The typing judgement . . . . .                              | 21        |
| <b>8</b> | <b>Operational semantics</b>                                | <b>21</b> |
| 8.1      | Projection . . . . .                                        | 21        |
| 8.2      | The three comparison methods . . . . .                      | 22        |
| 8.3      | Verdict evaluation . . . . .                                | 23        |
| 8.4      | Cost model . . . . .                                        | 23        |

|           |                                                              |           |
|-----------|--------------------------------------------------------------|-----------|
| <b>9</b>  | <b>Metatheory</b>                                            | <b>23</b> |
| 9.1       | Totality . . . . .                                           | 23        |
| 9.2       | Floor preservation . . . . .                                 | 24        |
| 9.3       | The refusal theorems . . . . .                               | 24        |
| 9.4       | What the metatheory does not give . . . . .                  | 25        |
| <b>10</b> | <b>Worked programs</b>                                       | <b>25</b> |
| 10.1      | Matched-filter motif detection . . . . .                     | 26        |
| 10.2      | Spectral homology search . . . . .                           | 26        |
| 10.3      | Four-column functional adjudication . . . . .                | 27        |
| 10.4      | An annotation transfer . . . . .                             | 28        |
| <b>11</b> | <b>The report</b>                                            | <b>29</b> |
| <b>12</b> | <b>Validation</b>                                            | <b>29</b> |
| 12.1      | What is checkable, and what is not . . . . .                 | 30        |
| 12.2      | Result . . . . .                                             | 30        |
| 12.3      | What the checking changed . . . . .                          | 30        |
| <b>13</b> | <b>The open problem, and what the language does about it</b> | <b>31</b> |
| 13.1      | The negative result . . . . .                                | 31        |
| 13.2      | The same failure on a second corpus . . . . .                | 31        |
| 13.3      | What is missing . . . . .                                    | 32        |
| 13.4      | What synopsis does about it . . . . .                        | 32        |
| <b>14</b> | <b>Related work</b>                                          | <b>32</b> |
| <b>15</b> | <b>Limitations</b>                                           | <b>33</b> |
| <b>16</b> | <b>Conclusion</b>                                            | <b>34</b> |

# 1 Introduction

## 1.1 Two failures of the same kind

Consider two questions a working geneticist asks daily.

- (i) **A synonymous variant.** A single base changes; the codon changes; the amino acid does not. Every content-based comparison reports the protein product as identical. Yet synonymous variants demonstrably alter translation kinetics, co-translational folding, mRNA stability and substrate specificity [46, 50, 33]. The comparison that reports *same* is not approximately right. It is reporting on a quantity that was never the one at issue.
- (ii) **Two unrelated enzymes catalysing one reaction.** Non-homologous isofunctional enzymes are common enough to have a name and a systematic census [41, 20]. Their sequences share nothing; every alignment reports maximal divergence. They do the same job. The comparison that reports *different* is, again, answering a different question than the one asked.

These are usually filed as separate difficulties — one a false positive, one a false negative — to be patched by better substitution matrices, deeper profile models, or structural prediction [24, 15, 29]. We will show they are one difficulty, and that it is structural rather than statistical.

The claim we prove is sharp. Let *content divergence* be any quantity computed from the two units in isolation, and let *response divergence* be a quantity computed from what each unit does to the network it sits in. Theorem 5.1 constructs a family in which content divergence is pinned at exactly zero — not small, exactly zero, at the theoretical minimum — while response divergence grows without bound. Theorem 5.4 constructs a family in which response divergence is exactly zero while content divergence grows. The two families lie on orthogonal axes of a two-dimensional plane. Any criterion returning a scalar computed from content alone projects that plane onto one axis, and necessarily maps one entire family to the origin. No refinement of the scalar repairs this; the defect is one of dimension, not of accuracy.

## 1.2 Sufficient alignment

The response to a comparison that cannot be exact is not to abandon comparison. It is to change what one asks of it.

There is an old and instructive precedent. The first three canonical gospels are called *synoptic* — seen together — because they can be laid out in parallel columns and read across [23, 58]. They are not identical. They differ in wording, in order, in detail, and in what each chooses to omit. A concordance demanding word-for-word agreement finds them in constant contradiction and produces nothing. The synoptic reading demands something weaker and far more useful: that the columns correspond *sufficiently* for a passage in one to be located against a passage in another, with the residue — what does not correspond — retained and displayed rather than discarded. The discipline is not to eliminate the difference. It is to bound it, to know where it lies, and to be able to say when it is small enough not to matter for the question at hand.

That is precisely the discipline genomic comparison needs and does not have. We take the name of our language from it. A *synopsis* is a parallel-column reading in which correspondence is certified, residue is carried, and the verdict is about sufficiency rather than identity.

The choice of name commits us to three design consequences, which we discharge in Section 6 and prove in Section 9.

- (a) **Residue is never discarded.** A comparison that returns a single number has thrown the residue away. In synopsis every value carries its residue in its type, and there is no constructor producing zero residue (Theorem 9.4).

- (b) **Exactness is not expressible.** If `identical` can be written, it will be written, and the two failures of Section 1 return. In synopsis it has no constructor (Theorem 9.5).
- (c) **The columns are structural.** A synopsis with one column is not a synopsis. The verdict constructor of synopsis takes four columns and omitting a pair is an arity error, not a permitted simplification (Theorem 9.6).

### 1.3 Where the invariant is not

Before building, we should say plainly what we take to be the source of the trouble, because it determines everything about the language’s treatment of sequence.

The tacit premise of content comparison is that whatever makes two units functionally the same is *written in them*, so that a sufficiently clever reading of the two strings recovers it. We claim this premise is false, and we can make the claim precise rather than rhetorical.

*Remark 1.1* (The invariant is not borne by the string). Theorem 2.7 shows that the quantity preserved under relabelling of a contact structure is the weight of a minimum cut, and Theorem 3.1 shows that this cut is in general a region of edges containing several vertices, not a single vertex. Theorem 2.8 then shows that vertex labels may be permuted arbitrarily without changing any separation cost. A sequence is a labelling. It is therefore, by Theorem 2.8, precisely the part of the structure that the invariant is indifferent to. Reading harder does not help because there is nothing further in the reading.

There is an elementary observation in the same direction that requires no theory at all. Every double-stranded nucleic acid carries two strands [63]. Read the complement in its own  $5' \rightarrow 3'$  direction and one obtains a different string, with a different codon decomposition, and different open reading frames. Both strands are physically present; both are read by the cell in different contexts; neither is privileged by the chemistry. If the functional invariant were a property of the string, the molecule would carry two of them and they would disagree. What the molecule actually determines is the *pairing* — the contact structure — and both strings are labellings of it. The invariant, if there is one, lives in the contacts.

Accordingly, in synopsis a sequence is an **opaque** type. It has no indexing operator, no substring operator, no base accessor, and no equality. It enters computation only by projection into a structure that is a legitimate bearer of the invariant. This is the single most restrictive decision in the language and Section 6.1 defends it in detail.

### 1.4 Why a language, and why a total one

One could implement all of this as a library. We give three reasons not to.

**First, a library cannot refuse.** The results of Section 5 say that certain comparisons are invalid — comparing content alone, comparing across incompatible receiver frames, reporting exact identity. In a library these are discouraged by documentation. In a language they can be made unrepresentable. That is the difference between a convention and a theorem, and Section 9 proves four theorems of exactly this shape. A refusal that the typechecker enforces is worth more than a paragraph in a manual that a hurried user does not read.

**Second, defaults are where science goes wrong.** A library function with a default threshold is an experiment with an undeclared parameter. Multiplied across a pipeline this is the mechanism by which analytic flexibility silently manufactures results [54, 21]. Synopsis has *no default thresholds anywhere*: every numeric parameter that can change a verdict must be written in the program text, and the report prints it. This costs the user keystrokes and buys the reader the ability to see what was chosen.

**Third, an experiment should terminate.** A laboratory protocol that might not finish is not a protocol. Synopsis is *total*: every well-typed program halts, and we prove it (Theorem 9.1). This is purchased by giving up general recursion and unbounded loops [60], which turns out

to cost nothing — none of the genomic procedures we survey needs them — and by requiring that the one genuinely iterative construct, relaxation, be given a step floor, at which point the compiler emits the iteration bound as a discharged obligation (Section 7.6).

This last point drives the one design decision we should flag explicitly because a reader may expect otherwise. Synopsis is an **analytic** language, not a generative one. It evaluates and certifies claims about given material. It does *not* search a space for sequences meeting a target specification. Generative search requires a solver whose termination depends on the geometry of the search space and cannot be established by typing; admitting it would forfeit Theorem 9.1, which is the strongest property the language has. We judge the trade correct: a total language that certifies is more useful to an experimentalist than a partial one that also proposes, and the generative layer, if wanted, can be built above synopsis and made to emit synopsis programs for certification.

## 1.5 Contributions and organisation

- Section 2 develops contact structures and proves the contact floor (Theorem 2.4), the invariance of cut weight (Theorem 2.7), and its conditional survival under expansion (Theorem 2.13), with an explicit counterexample (Theorem 2.10) showing the unconditional statement to be *false*.
- Section 3 proves that identity is regional (Theorem 3.1), that labels are free (Theorem 2.8), and that registered value is receiver-relative (Theorem 3.5) with disagreement that is not noise (Theorem 3.6).
- Section 4 builds cross-demand and relaxation and proves monotonicity (Theorem 4.9), the termination dichotomy (Theorem 4.10) and the form of quiescence (Theorem 4.11).
- Section 5 proves the two separation results (Theorem 5.1, Theorem 5.4) and derives the four-column criterion (Theorem 5.10).
- Section 6 states the eleven design rules of synopsis and argues each from a result above.
- Section 7 gives the concrete language: lexis, grammar, types and the typing judgement.
- Section 8 gives the operational semantics.
- Section 9 proves the metatheory: totality, floor preservation, scope safety, and the four refusal theorems.
- Section 10 gives worked `.syp` programs for three families of experiment.
- Section 11 specifies the report and its certificates.
- Section 13 reports the negative result on response independence and what the language does about it.

## 2 Contact structures and the floor

We build the theory from nothing. A reader who wants only the language may read Section 6 onwards and refer back, but the design rules are consequences of these results and are arbitrary without them.

## 2.1 Finite weighted graphs

**Definition 2.1** (Finite weighted graph). A *finite weighted graph* is a triple  $\mathcal{G} = (V, E, w)$  with  $V$  a finite non-empty set,  $E \subseteq \binom{V}{2}$ , and  $w : E \rightarrow \mathbb{R}_{>0}$ . Weights are *strictly* positive: an edge of weight zero is not a light edge, it is the absence of an edge, and we insist on the distinction because every result below turns on it.

**Definition 2.2** (Cut). For  $S \subseteq V$  the *cut* induced by  $S$  is  $\partial S = \{e \in E : |e \cap S| = 1\}$  with weight  $\text{cut}(S) = \sum_{e \in \partial S} w(e)$ . For  $u \neq v$  a set  $S$  is  $(u, v)$ -*separating* if  $u \in S$  and  $v \notin S$ , and

$$\sigma(u \mid v) = \min\{\text{cut}(S) : S \text{ is } (u, v)\text{-separating}\}.$$

The minimum is over a finite non-empty collection, hence attained.

We call  $\sigma(u \mid v)$  the *separation cost*: the cheapest way, in the currency the weights supply, to place  $u$  and  $v$  on opposite sides of a boundary. By the max-flow min-cut theorem it equals the maximum flow from  $u$  to  $v$  [17], and is computable in polynomial time [57, 31]; all-pairs values admit a Gomory–Hu tree [22]. We will not need efficiency for the theory, but the language’s cost model does (Section 8.4).

**Definition 2.3** (Contact structure). A *contact structure* is a finite weighted graph with a distinguished vertex  $\mathbf{m} \in V$ , the *medium*, adjacent to every other vertex. Vertices other than  $\mathbf{m}$  are *items*, written  $I = V \setminus \{\mathbf{m}\}$ . We write  $\sigma(v) = \sigma(v \mid \mathbf{m})$  for the *separation cost of an item*,  $\Omega = \sum_{e \in E} w(e)$  for the total weight, and

$$\beta(\mathcal{G}) = \min_{e \in E} w(e)$$

for the *contact floor*.

The medium formalises the background against which anything is individuated. Every distinction is a distinction *from* something;  $\mathbf{m}$  is that something, and its universal adjacency says the background is always available and never absent. This is the graph-level statement of a very old idea — that a thing is known by what it is not — and it is what makes  $\sigma(v)$  the cost of individuating  $v$  at all, rather than merely of separating it from a chosen rival.

## 2.2 The floor

**Theorem 2.4** (Contact floor). *Let  $\mathcal{G}$  be a contact structure and  $v$  an item. Then*

$$\sigma(v) \geq \beta(\mathcal{G}) > 0.$$

*Proof.*  $E$  is finite and non-empty (the medium is adjacent to every item and  $I \neq \emptyset$ ), and  $w$  takes values in  $\mathbb{R}_{>0}$ , so  $\beta(\mathcal{G}) = \min_e w(e)$  is the minimum of a finite set of strictly positive reals and is therefore strictly positive.

Let  $S$  be any  $(v, \mathbf{m})$ -separating set. Since  $v \in S$  and  $\mathbf{m} \notin S$  and  $\{v, \mathbf{m}\} \in E$  by adjacency of the medium, we have  $|\{v, \mathbf{m}\} \cap S| = 1$ , so  $\{v, \mathbf{m}\} \in \partial S$ . Hence  $\partial S \neq \emptyset$  and  $\text{cut}(S) \geq w(\{v, \mathbf{m}\}) \geq \beta(\mathcal{G})$ . Taking the minimum over separating  $S$  gives  $\sigma(v) \geq \beta(\mathcal{G})$ .  $\square$

The proof is three lines and the content is substantial: *no distinction is free*. There is a strictly positive price on telling any two things apart, fixed by the cheapest contact in the structure. Two items may be as similar as the structure permits — they may sit exactly at the floor — but they cannot be at zero, because zero is not in the range of  $w$ .

This is the formal reason a comparison should never return **identical**. Identity in the sense of *indistinguishable at cost zero* does not occur in a contact structure. Reporting it is reporting a value outside the range of the quantity being measured.

**Definition 2.5** (Local floor). For an item  $v$ ,  $\beta_k(v) = \min\{w(e) : v \in e\}$ , the minimum weight incident to  $v$ .

Clearly  $\beta(\mathcal{G}) \leq \beta_k(v)$  for every  $v$ , and  $\beta(\mathcal{G}) = \min_v \beta_k(v)$ .

### 2.3 Invariance under relabelling

**Definition 2.6** (Weighted isomorphism). A *weighted isomorphism* of contact structures  $\mathcal{G} \rightarrow \mathcal{G}'$  is a bijection  $h : V \rightarrow V'$  with  $h(\mathbf{m}) = \mathbf{m}'$ ,  $\{h(u), h(v)\} \in E' \iff \{u, v\} \in E$ , and  $w'(\{h(u), h(v)\}) = w(\{u, v\})$ .

**Theorem 2.7** (Cut weight is the invariant). *If  $h : \mathcal{G} \rightarrow \mathcal{G}'$  is a weighted isomorphism then for all items  $u, v$ ,*

$$\sigma'(h(u) \mid h(v)) = \sigma(u \mid v), \quad \sigma'(h(v)) = \sigma(v), \quad \beta(\mathcal{G}') = \beta(\mathcal{G}).$$

*Proof.*  $h$  induces a bijection  $S \mapsto h(S)$  between subsets of  $V$  and of  $V'$ , carrying  $(u, v)$ -separating sets to  $(h(u), h(v))$ -separating sets bijectively. It carries  $\partial S$  to  $\partial h(S)$  edge for edge and preserves each weight, so  $\text{cut}'(h(S)) = \text{cut}(S)$ . Minimising over a bijectively-corresponding collection of equal values gives the first identity; the second is the case  $v = \mathbf{m}$ , using  $h(\mathbf{m}) = \mathbf{m}'$ ; the third holds because  $h$  is a weight-preserving bijection on  $E$ .  $\square$

**Corollary 2.8** (Labels are free). *Let  $\mathcal{G}$  be a contact structure and  $\rho$  any permutation of the items. Let  $\mathcal{G}^\rho$  be  $\mathcal{G}$  with items renamed by  $\rho$ . Then every separation cost, every minimum cut weight and the floor are unchanged.*

*Proof.*  $\rho$  extended by  $\mathbf{m} \mapsto \mathbf{m}$  is a weighted isomorphism  $\mathcal{G} \rightarrow \mathcal{G}^\rho$ . Apply Theorem 2.7.  $\square$

Theorem 2.8 is the formal content of Theorem 1.1. The names carry *no* part of the invariant — not a small part, none. A comparison procedure that reads names is reading the one component of the structure guaranteed to be irrelevant.

### 2.4 Expansion, and a false theorem made true

Real structures are never observed complete. One always has a subgraph and may later add more. We must ask whether the floor survives.

**Definition 2.9** (Expansion). An *expansion* of  $\mathcal{G}$  is a contact structure  $\mathcal{G}^+$  with  $V \subseteq V^+$ ,  $E \subseteq E^+$ ,  $\mathbf{m}^+ = \mathbf{m}$  and  $w^+|_E = w$ . An *expansion sequence* is a chain  $\mathcal{G}_0 \subseteq \mathcal{G}_1 \subseteq \dots$ .

It is tempting to assert that separation costs stay bounded below along any expansion sequence. They do not.

*Remark 2.10* (The unconditional claim is false). Fix an item  $v$  and let  $\mathcal{G}_k$  add to  $\mathcal{G}_{k-1}$  a fresh vertex  $x_k$  joined to  $v$  by an edge of weight  $1/k$  (and to  $\mathbf{m}$  at weight 1). Then  $\beta(\mathcal{G}_k) \leq 1/k \rightarrow 0$ , and the local floor  $\beta_k(v) \rightarrow 0$ . With weights  $1/k^2$  the sum  $\sum_k 1/k^2$  converges, so one may arrange that  $\sigma(v)$  itself is driven towards a limit determined entirely by the new light edges:  *$v$  dissolves into the medium*. In the limit no positive lower bound survives. Any statement that the floor is unconditionally preserved under expansion is therefore false, and we record the counterexample rather than the temptation.

*Remark 2.11* (Measured decay). Running the three sequences for 40 steps from a common start makes the separation concrete: the  $1/k$  expansion drives  $\beta_k(v)$  to 0.025, the  $1/k^2$  expansion to  $6.25 \times 10^{-4}$ , while a floor-stabilising expansion holds it at 1.0 exactly (Section 12, E06). The gap is three orders of magnitude after forty steps and widens without limit; the hypothesis of Theorem 2.12 is doing real work, not tidying an edge case.

The correct statement needs a hypothesis, and naming it precisely is the point.

**Definition 2.12** (Floor-stabilising expansion). An expansion sequence  $(\mathcal{G}_k)$  is *floor-stabilising* at  $v$  if there is  $\beta_* > 0$  with  $\beta_k(v) \geq \beta_*$  for all  $k$ : the newly added contacts at  $v$  do not become arbitrarily cheap.



**Theorem 2.13** (Conditional invariance under expansion). *Let  $(\mathcal{G}_k)$  be an expansion sequence that is floor-stabilising at  $v$  with constant  $\beta_*$ . Then  $\sigma_k(v) \geq \beta_*$  for all  $k$ , and the sequence  $(\sigma_k(v))$  is bounded below by a positive constant independent of  $k$ .*

*Proof.* Fix  $k$  and let  $S$  be  $(v, \mathbf{m})$ -separating in  $\mathcal{G}_k$ . As in Theorem 2.4,  $\{v, \mathbf{m}\} \in \partial S$ , so  $\text{cut}_k(S) \geq w_k(\{v, \mathbf{m}\}) \geq \beta_k(v) \geq \beta_*$ . Minimise over  $S$ .  $\square$

*Remark 2.14.* Theorem 2.13 is weaker than one would like and exactly as strong as is true. The hypothesis is checkable — it is a statement about the incident weights actually observed — and a language can require the user to assert it and can record the assertion in the report. Synopsis does: see Section 7.4 and the **require** form.

## 2.5 Perturbation

**Definition 2.15** (Perturbation). A *perturbation* of  $\mathcal{G}$  is a map  $\delta : E \rightarrow \mathbb{R}_{\geq 0}$ ; it acts by  $w \mapsto w + \delta$ . Perturbations are non-negative: they raise the cost of distinctions and never create separation for free.

**Proposition 2.16** (Skeleton invariance). *Under any perturbation,  $V$  and  $E$  are unchanged, every separation cost is non-decreasing, and if  $\delta$  vanishes on a minimum-weight edge then  $\beta$  is unchanged.*

*Proof.* The vertex and edge sets do not appear in Theorem 2.15, so they are fixed. For any  $S$ ,  $\text{cut}_{w+\delta}(S) = \text{cut}_w(S) + \sum_{e \in \partial S} \delta(e) \geq \text{cut}_w(S)$ ; minimising both sides over the same finite collection preserves the inequality. If  $\delta(e_0) = 0$  for some  $e_0$  attaining the minimum weight, then  $\min_e (w + \delta)(e) \leq w(e_0) = \beta$ , and  $\geq$  holds since  $\delta \geq 0$ .  $\square$

The distinction matters and is easy to lose: perturbation changes the *costs* without changing the *contacts*. A unit which perturbs its surroundings does not rewire them; it makes some distinctions in them dearer. This is the sense in which a variant has an effect without editing the network.

## 3 Identity, region, and the receiver

### 3.1 Identity is a region, not a point

Theorem 2.7 identifies the invariant as a cut weight. We now show the minimising *set* is generally not a singleton, so the invariant is borne by a region.

**Theorem 3.1** (Regionality). *There exist contact structures in which, for an item  $v$ , every minimising  $(v, \mathbf{m})$ -separating set has cardinality at least 3.*

*Proof.* Fix  $r \geq 3$  and  $\beta > 0$  with  $\beta$  small. Take items  $A = \{a_1, \dots, a_r\}$  and  $B = \{b_1, \dots, b_r\}$ . Join every pair within  $A$  and every pair within  $B$  by an edge of weight  $W > 0$ ; join  $a_1$  to  $b_1$  by a single edge of weight  $\beta$ ; join every item to  $\mathbf{m}$  by an edge of weight  $\beta$ . Let  $v = a_1$ .

Consider  $S = \{a_1\}$ . Its cut severs the  $r - 1$  edges from  $a_1$  to the rest of  $A$ , the bridge to  $b_1$ , and the medium edge, so

$$\text{cut}(\{a_1\}) = (r - 1)W + \beta + \beta = (r - 1)W + 2\beta.$$

Now consider  $S = A$ . Its cut severs the bridge and the  $r$  medium edges:

$$\text{cut}(A) = \beta + r\beta = (r + 1)\beta,$$

which is independent of  $W$ . Whenever  $(r - 1)W + 2\beta > (r + 1)\beta$ , i.e.  $W > \beta \cdot \frac{r-1}{r-1} = \beta$ , we get  $\text{cut}(A) < \text{cut}(\{a_1\})$ . More generally, a separating set  $S$  with  $a_1 \in S$  and  $|S \cap A| = j < r$  severs

at least  $j(r-j)$  intra- $A$  edges of weight  $W$  plus  $j$  medium edges, giving  $\text{cut}(S) \geq j(r-j)W + j\beta$ , which for  $1 \leq j \leq r-1$  is at least  $(r-1)W + \beta$  and so exceeds  $(r+1)\beta$  once  $W > 2\beta$ . Taking  $S \supseteq A$  and adding any  $b_i$  only adds intra- $B$  edges of weight  $W$  and further medium edges, strictly increasing the cut unless  $S \supseteq A \cup B$ , which is not separating from  $\mathbf{m}$ 's side only if  $\mathbf{m} \notin S$  — and then  $\text{cut}(A \cup B) = 2r\beta > (r+1)\beta$  for  $r > 1$ .

Hence for  $W > 2\beta$  and  $r \geq 3$  the unique minimiser is  $S^* = A$ , of cardinality  $r \geq 3$ .  $\square$

*Remark 3.2* (A witness we got wrong first). Our first attempt at Theorem 5.1 used the informal principle “if two items are joined by an edge of weight  $\beta$  then they are separated at the floor”. This is false, and Theorem 3.1 is why: the minimum cut separating  $a_1$  from  $b_1$  in the construction above is *not* the bridge, because detaching the whole cluster  $A$  may be cheaper or dearer depending on  $W$ , and for moderate  $W$  neither is the bridge alone. Computing the cut exactly rather than reasoning from the light edge gave 1.4 where the informal argument predicted 0.2. We record the episode because the informal principle is intuitive, wrong, and would have propagated silently into the language’s semantics.

**Corollary 3.3.** *The invariant of Theorem 2.7 is not in general attributable to any single vertex, and a fortiori not to any single label.*

### 3.2 Receiver-relativity

The value an item registers is not a property of the item.

**Definition 3.4** (Receiver). A *receiver* is a contact structure  $N$  together with a designated embedding site. A unit  $u$  *registers* in  $N$  the value  $\sigma_N(u)$ , computed after  $u$  is embedded at the site.

**Theorem 3.5** (Receiver-relativity). *There exist a unit  $u$  and receivers  $N, N'$  differing in a single edge weight, with  $\sigma_N(u) \neq \sigma_{N'}(u)$ . Consequently  $\sigma(u)$  is a function of the pair  $(u, N)$  and not of  $u$  alone.*

*Proof.* Take  $N$  with items  $u, x, y$ , edges  $\{u, x\}$  and  $\{x, y\}$  of weight 1, medium edges of weight 1, and  $\{u, y\}$  absent. Then a  $(u, \mathbf{m})$ -separating set must sever  $\{u, \mathbf{m}\}$  and  $\{u, x\}$  unless it contains  $x$ , so  $\sigma_N(u) = \min\{1+1, 1+1+1\} = 2$  where the second option is  $S = \{u, x\}$  severing  $\{u, \mathbf{m}\}, \{x, \mathbf{m}\}, \{x, y\}$ . Now let  $N'$  differ only in  $w(\{x, y\}) = \frac{1}{4}$ . Then  $S = \{u, x\}$  costs  $1+1+\frac{1}{4} = 2.25$  and  $S = \{u\}$  still costs 2, so  $\sigma_{N'}(u) = 2$ . Instead set  $w(\{u, x\}) = \frac{1}{4}$  in  $N'$ : then  $S = \{u\}$  costs  $1 + \frac{1}{4} = 1.25 < 2$ , so  $\sigma_{N'}(u) = 1.25 \neq 2 = \sigma_N(u)$ , and  $N, N'$  differ in exactly one weight.  $\square$

**Corollary 3.6** (Disagreement is not noise). *Let  $N, N'$  be receivers with  $\sigma_N(u) \neq \sigma_{N'}(u)$ . Then no increase in the precision or number of measurements performed in either receiver reduces the discrepancy, since each value is exactly determined by its own structure. The two receivers are evaluating different functions and each is correct.*

*Remark 3.7* (How often, and why the rate is a minority). It would be a misreading of Theorem 3.5 to expect that changing the receiver usually changes the value. It does not. Perturbing a single background edge — one not incident to  $u$  — moves  $\sigma(u)$  only when that edge crosses  $u$ 's minimising cut, and most edges do not. Sweeping a perturbation across every non-incident edge of a population of random contact structures, 206 of 1181 edges (17.4%) moved  $\sigma(u)$  at all; where an edge did move it, the smallest displacement recorded was 0.022. Independently, over 150 receiver pairs the value disagreed in 26 cases (17.3%), and where it disagreed the discrepancy was never small — the smallest was 0.056, five orders of magnitude above numerical tolerance (Section 12, E10).

Both halves of this matter. That the rate is a minority is why receiver-relativity is easy to overlook in practice and easy to misattribute to noise when it appears. That the discrepancies are never marginal is what makes Theorem 3.6 substantive. The observed displacements are broadly spread rather than concentrated, but they are bounded away from zero: either the receivers agree exactly, or they disagree by an amount no measurement programme will close. There is no population of near-misses in between, which is precisely what a noise account would predict.

Theorem 3.6 has an immediate practical reading and a language consequence. The practical reading is that a variant reported with different effect in two assays need not indicate that one assay is wrong; the assays may be different receivers, both right. The language consequence is that values from different receivers must not be compared, and synopsis enforces this lexically: values are scoped to their frame and cross-frame comparison is a *scope error*, not a runtime warning (Section 7.4, Theorem 9.7).

## 4 Blind comparison: demand, relaxation, quiescence

We now build a comparison that certifies correspondence without reading what anything means. This is essential: any procedure that must read meaning inherits the annotation errors already endemic in the databases it reads [51].

### 4.1 Alignment and distance to the floor

**Definition 4.1** (Alignment). For items  $u, v$  of a contact structure  $\mathcal{G}$ ,

$$a(u, v) = \frac{\sigma(u \mid v)}{\Omega}, \quad \frac{\beta}{\Omega} \text{ the floor ratio}, \quad \Delta(u, v) = a(u, v) - \frac{\beta}{\Omega}.$$

$\Delta$  is the *distance to the floor*.

**Proposition 4.2.**  $\Delta(u, v) \geq 0$  always, with equality only if  $u$  and  $v$  are separated exactly at the floor.

*Proof.* Immediate from Theorem 2.4 applied to the pair  $(u, v)$ : any separating set severs at least one edge, of weight at least  $\beta$ .  $\square$

Normalising by  $\Omega$  makes  $a$  dimensionless and comparable across structures of different total weight; subtracting the floor ratio makes  $\Delta$  vanish exactly when the pair is as close as the structure permits. *Sufficient* alignment will mean  $\Delta$  small; it will never mean  $a = 0$ , which Theorem 2.4 forbids.

### 4.2 Anchors and residue

Not every item in a structure is equally load-bearing for a given item’s individuation. We split by ablation.

**Definition 4.3** (Ablation score, anchors, residue). For items  $x \neq u$ , the *ablation score* is  $\alpha(x; u) = \sigma_{\mathcal{G}}(x) - \sigma_{\mathcal{G} \setminus u}(x)$ , the drop in  $x$ ’s separation cost when  $u$  is deleted. Fix  $r \geq 0$ . The *anchors* of  $x$  are the  $r$  items of largest  $\alpha(x; \cdot)$ , ties broken by a fixed total order on names; the *residue*  $\varrho(x)$  is the remainder.

The intuition is that anchors are what  $x$  is held in place by, and residue is everything else — the part of the structure whose correspondence still has to be established. The tie-breaking by a fixed order is not cosmetic: without it the split is not a function, and a language that intends to be reproducible cannot afford a nondeterministic primitive. Synopsis therefore fixes the order and the report prints  $r$ .

### 4.3 Cross-demand

**Definition 4.4** (Cross-demand). Let  $\mathcal{G}_A, \mathcal{G}_B$  be contact structures and  $\varphi : I_B \rightarrow I_A$  a partial correspondence. The demand from  $A$  on  $B$  is

$$D_{A \rightarrow B} = \sum_{i \in \mathcal{Q}_B} \max\left(0, a_B(i, \varphi(i)) - \frac{\beta_B}{\Omega_B}\right), \quad (1)$$

the sum taken over residue items of  $B$  on which  $\varphi$  is defined.

Each term is a distance to the floor and is non-negative by Theorem 4.2; the clamp guards only floating-point noise. The quantity is a *demand* in the sense that it measures how much reconciliation  $B$  still requires of  $A$ : it is zero exactly when every residue item of  $B$  is at the floor from its correspondent.

Crucially, (1) reads no labels, no annotations, no sequence. It is computed from cuts. That is what makes it blind.

**Definition 4.5** (Total demand).  $D = D_{A \rightarrow B} + D_{B \rightarrow A}$ , using  $\varphi$  and its inverse.

Bidirectionality is not decoration. A one-sided demand can be made small by choosing a correspondence that maps everything difficult to undefined; the reverse direction penalises exactly that.

### 4.4 Relaxation

**Definition 4.6** (Relaxation). Let  $D_0 \geq 0$ . A *relaxation* is a sequence  $D_0 > D_1 > \dots$  produced by an update  $U$  with  $D_{k+1} = U(D_k)$ , terminating when  $D_k \leq \theta$  for a stated threshold  $\theta \geq 0$ . The relaxation *declines* at step  $k$  if  $U(D_k)$  is undefined — no effective update exists.

**Assumption 4.7** (Effective update). There is  $\eta > 0$  such that whenever  $U(D)$  is defined, either

$$U(D) \leq D - \eta \quad \text{or} \quad U(D) \leq \theta.$$

*Remark 4.8* (Why the second disjunct is not redundant). The bare condition  $U(D) \leq D - \eta$  is what the termination argument needs and is all Theorem 4.9 uses, but it cannot be imposed as a runtime check: on the final step, when  $D < \eta$ , an update that lands on quiescence lands on a value that is not below the negative quantity  $D - \eta$ . A checker enforcing the first disjunct unconditionally reports a violation on the last step of every terminating run — which is what ours did before we separated the two cases. The disjunctive form is the assumption that is simultaneously strong enough for Theorem 4.10 and enforceable, and it is the form the language checks.

Theorem 4.7 is the whole of what makes relaxation a terminating procedure, and it is a real assumption, not a technicality: an update permitted to make arbitrarily small progress can run forever while always improving. Synopsis therefore refuses to typecheck a relaxation whose  $\eta$  is not bounded below by a written constant (Section 7.6).

**Theorem 4.9** (Monotone decrease). *Under Theorem 4.7 the sequence  $(D_k)$  is strictly decreasing while defined, and  $D_k \leq D_0 - k\eta$ .*

*Proof.* Induction:  $D_0 \leq D_0 - 0$ ; if  $D_k \leq D_0 - k\eta$  and  $U(D_k)$  is defined then, since the loop has not terminated we have  $D_k > \theta$ , so the second disjunct of Theorem 4.7 would end the run and the bound is asserted only where the first applies:  $D_{k+1} \leq D_k - \eta \leq D_0 - (k+1)\eta$ .  $\square$

**Theorem 4.10** (Termination dichotomy). *Under Theorem 4.7, exactly one of the following occurs within  $\lceil D_0/\eta \rceil$  steps:*

- (i) **quiescence:**  $D_k \leq \theta$  for some  $k \leq \lceil D_0/\eta \rceil$ ; or
- (ii) **decline:**  $U(D_k)$  is undefined for some  $k \leq \lceil D_0/\eta \rceil$ .

There is no third outcome, and in particular the relaxation cannot diverge or cycle.

*Proof.* Suppose neither (i) nor (ii) occurs for any  $k \leq K = \lceil D_0/\eta \rceil$ . Then  $U$  is defined at every such step and  $D_k > \theta \geq 0$  for all  $k \leq K$ . By Theorem 4.9,  $D_K \leq D_0 - K\eta \leq D_0 - D_0 = 0$ , contradicting  $D_K > \theta \geq 0$ . Exclusivity holds because (i) terminates the sequence and (ii) is the failure of the update that would continue it.  $\square$

Theorem 4.10 is the theorem that lets synopsis be total while still containing a loop. The bound  $\lceil D_0/\eta \rceil$  is computable from values in scope at the point the loop is written, so the compiler can emit it. Section 7.6 does exactly that.

**Theorem 4.11** (Form of quiescence). *Quiescence at  $\theta = 0$  means every residue term of (1) is zero, i.e. every corresponding residue pair is separated exactly at the floor. It does not mean any pair is indistinguishable: by Theorem 2.4 each such pair still costs  $\beta > 0$  to separate.*

*Proof.* A finite sum of non-negative terms is zero iff each term is zero; each term is  $\Delta(i, \varphi(i))$ , which vanishes iff the pair is at the floor (Theorem 4.2). The second statement is Theorem 2.4.  $\square$

Theorem 4.11 is the formal statement of *sufficient alignment*. Two structures may place no demand on one another and remain fully distinguishable. This is the synoptic reading made precise: the columns correspond; the residue is positive; both facts are reported.

## 5 The separation results and the four-column criterion

### 5.1 Content-aligned, response-divergent

**Theorem 5.1** (False friends). *There is a family of pairs  $(u_c, v_c)$ , indexed by  $c \geq 0$ , embedded in contact structures, such that*

$$D^{\text{central}}(u_c, v_c) = 0 \quad \text{for all } c, \quad \text{while} \quad D^{\text{response}}(u_c, v_c) \longrightarrow \text{unbounded as } c \rightarrow \infty.$$

*That is: content is aligned exactly at the floor for every  $c$ , while the effect on the surrounding network diverges.*

*Proof. Construction.* Let  $P$  be a contact structure containing two pendant items  $u, v$ , each joined to the medium by an edge of weight  $\beta$  and to nothing else. Then any  $(u, v)$ -separating set may take  $S = \{u\}$ , severing exactly  $\{u, \mathbf{m}\}$ , so  $\sigma(u \mid v) = \beta$  and  $\Delta(u, v) = 0$ . Being pendant is essential; by Theorem 3.2 a light edge between non-pendant items does *not* suffice.

Take  $\mathcal{G}_A$  and  $\mathcal{G}_B$  each equal to  $P$ , with  $\varphi$  the identity on the pendants. All residue terms are  $\Delta = 0$ , so  $D^{\text{central}} = 0$  for every  $c$ : content comparison reports correspondence unconditionally.

*Response.* Let the receiver  $N$  be  $P$  itself, padded with a fixed block of  $n_f$  filler items mutually joined at weight  $w_f$  and each joined to the medium at  $w_f$ , with  $w_f > \beta$ . Let the response perturbation  $\delta_c$  raise *both* pendant edges  $\{u, \mathbf{m}\}$  and  $\{v, \mathbf{m}\}$  by  $c$ . By Theorem 2.16 the skeleton is unchanged, so  $N$  and  $N_c$  are comparable edge for edge under the identity correspondence.

Both pendants must be raised. If only one is raised, the minimum  $(u, v)$ -cut is realised by a separating set that places the medium on the raised side and severs only the untouched pendant edge; its weight is  $\beta$  for every  $c$ , and  $\Delta$  does not move at all. This is not a defect of the bound but of the witness, and it is the reason the one-sided construction — the natural first attempt — registers no response whatever.

With both raised, every  $(u, v)$ -separating set severs at least one edge of weight  $\beta + c$ , so  $\sigma_{N_c}(u \mid v) = \beta + c$ , while  $\Omega_c = \Omega_0 + 2c$  and the floor is  $\beta_c = \min\{\beta + c, w_f\}$ . Hence

$$\Delta_c(u, v) = \frac{\beta + c}{\Omega_0 + 2c} - \frac{\min\{\beta + c, w_f\}}{\Omega_0 + 2c} = \frac{\max\{0, \beta + c - w_f\}}{\Omega_0 + 2c}, \quad (2)$$

which is identically 0 for  $c \leq w_f - \beta$  and thereafter strictly increasing to the limit  $\frac{1}{2}$ . The flat initial segment is Theorem 2.4 operating correctly rather than an absent response: while the pendant edges are still the lightest in the structure they *are* the floor, so  $\beta_c$  rises in lockstep with the cut and the difference cannot open. The response becomes visible exactly once the pendants exceed the surrounding weight  $w_f$ .

The divergence is therefore in the normalised gap, which is bounded by  $\frac{1}{2}$  on any single pendant pair; to obtain an unbounded  $D^{\text{response}}$  it suffices to take a residue of  $m$  independent pendant pairs, each contributing a term of (2), so that  $D^{\text{response}} \geq m \Delta_c \rightarrow \infty$  with  $m$ . What the theorem requires, and what (2) supplies, is that the response separates without bound while the central demand is pinned at zero.  $\square$

*Remark 5.2 (Measured).* On the instance  $\beta = 0.2$ ,  $n_f = 3$ ,  $w_f = 1$ , sweeping  $c$  over  $[1, 5]$  gives  $\Delta_c$  rising from 0.0238 to 0.2561 while  $D^{\text{central}}$  is 0 to machine precision throughout (Section 12, E16). Below  $c = w_f - \beta$  the response is exactly 0, as (2) predicts.

*Remark 5.3.* The biological instance is (i) of Section 1. Two units whose content a comparison cannot separate — a synonymous pair — may act on the cell in arbitrarily different ways [33, 50]. The theorem says this is not an edge case to be handled but a structural possibility that content comparison is blind to by construction.

## 5.2 Content-divergent, response-aligned

**Corollary 5.4 (Converse).** *There is a family indexed by a bridge weight  $d \geq 0$  with  $D^{\text{central}}$  strictly increasing in  $d$  over an initial interval, and  $D^{\text{response}} = 0$  for all  $d$ .*

*Proof.* Take the two-cluster structure of Theorem 3.1: two  $r$ -cliques at intra-weight  $W$ , joined by the single bridge  $\{a_0, b_0\}$ , every item joined to the medium at  $\beta$ . Put  $u = a_1$ ,  $v = b_1$  and index the family by the bridge weight  $d$ , so that the bridge carries  $\beta + d$ .

The index must be the bridge, not  $W$ . Raising  $W$  leaves the minimum  $(a_1, b_1)$ -cut fixed — each of  $a_1, b_1$  detaches through the medium at  $\beta$  regardless of how heavy its own clique becomes — while  $\Omega = r(r-1)W + \beta + 2r\beta$  grows linearly in  $W$ . So  $a(u, v)$  and hence  $\Delta$  *fall* as  $W$  increases. The bridge is the edge that the  $(a_1, b_1)$ -cut actually crosses, and it is therefore the parameter the central demand responds to.

With  $d$  as index, any  $(a_1, b_1)$ -separating set must either sever  $a_1$  from the medium and from its clique, or cross the bridge; in the regime where the latter is minimal the cut weight is affine in  $d$  with positive coefficient while  $\Omega$  grows at the same absolute rate on a single edge. Since the cut is a minimum of finitely many affine functions of  $d$ ,  $a(u, v)$  is piecewise affine and strictly increasing until the minimiser switches to a  $d$ -independent set, after which it is non-increasing. Restricting to the initial interval gives strict increase, as claimed.

Let both  $u$  and  $v$  induce the *same* perturbation  $\delta$  on the receiver. Then  $N_u = N_v$  identically, every  $\Delta$  term is computed on one structure against itself under the identity correspondence, and  $D^{\text{response}} = 0$  exactly.  $\square$

*Remark 5.5 (Measured).* At  $r = 4$ ,  $W = 1$ ,  $\beta = 0.2$ , sweeping the bridge  $d$  over  $[0, 2]$  raises  $\Delta(a_1, b_1)$  monotonically from 0.0580 to 0.1772, with  $D^{\text{response}} = 0$  throughout (Section 12, E17). Sweeping  $W$  instead over  $[0.5, 6]$  sends  $\Delta$  from 0.1026 *down* to 0.0108, with the cut constant at 1.0 and  $\Omega$  growing from 7.8 to 73.8 — which is why the corollary is indexed as it is. Continuing the bridge sweep past  $d \approx 2$  turns the curve over as the minimiser switches, so the strict increase is genuinely local, not global.

*Remark 5.6.* This is (ii) of Section 1: the non-homologous isofunctional enzymes [41]. Content comparison reports maximal divergence; the response columns report correspondence.

### 5.3 The plane and the criterion

**Corollary 5.7** (Orthogonality). *The families of Theorem 5.1 and Theorem 5.4 lie on the two coordinate axes of the plane  $(D^{\text{central}}, D^{\text{response}})$ : the first on  $\{0\} \times \mathbb{R}_{\geq 0}$ , the second on  $\mathbb{R}_{\geq 0} \times \{0\}$ . Any criterion that is a function of  $D^{\text{central}}$  alone is constant on the first family and therefore cannot separate its members; any criterion that is a function of  $D^{\text{response}}$  alone is constant on the second.*

*Remark 5.8* (Measured). Both axes are occupied strictly, not merely approached. Over the both-pendant family of Theorem 5.2 at  $c \in [1.5, 5]$ , every member has  $D^{\text{central}} = 0$  to machine precision and  $D^{\text{response}} > 0$  strictly; over the bridge family of Theorem 5.5 at  $d \in [0.25, 2]$ , every member has  $D^{\text{response}} = 0$  and  $D^{\text{central}} > 0$  strictly (Section 12, E18). The stronger per-axis statement matters: a family that merely avoided the origin would be consistent with the two demands being correlated, which is what Theorem 5.9 is built to exclude.

This forces the shape of a correct criterion.

**Definition 5.9** (Four columns). *A four-column comparison consists of two central columns  $(C_A, C_B)$  — the units under comparison, with their anchors and residue — and two response columns  $(R_A, R_B)$  — the receivers as perturbed by each unit — together with correspondences  $\varphi^c$  and  $\varphi^r$  and a threshold  $\theta > 0$ . Its verdict is*

$$\text{verdict} = \begin{cases} \text{CORRESPOND} & D^c \leq \theta \text{ and } D^r \leq \theta, \\ \text{DECLINE} & \text{some relaxation declined (Theorem 4.10(ii))}, \\ \text{DIVERGE} & \text{otherwise.} \end{cases}$$

**Theorem 5.10** (Adequacy of four columns). *The four-column verdict separates the families of Theorem 5.1 and Theorem 5.4, and no criterion depending on fewer than both demand coordinates does.*

*Proof.* On the first family  $D^c = 0 \leq \theta$  but  $D^r$  exceeds  $\theta$  for  $c$  large, so the verdict is DIVERGE, correctly. On the second  $D^r = 0 \leq \theta$  but  $D^c$  exceeds  $\theta$  for  $W$  large, so again DIVERGE — and at small  $W$ , CORRESPOND, tracking the genuine functional agreement. The impossibility half is Theorem 5.7.  $\square$

*Remark 5.11* (Why DECLINE is a verdict). A three-valued verdict is not a hedge. Theorem 4.10 gives exactly two ways a relaxation can end, and one of them is the discovery that no effective update exists — the comparison, run correctly to its end, does not decide. Collapsing that onto DIVERGE reports a finding where there is none, and collapsing it onto CORRESPOND is worse. On a laboratory bench “this experiment does not decide” is a result one writes down. Synopsis makes DECLINE a first-class inhabitant of verdict for that reason.

### 5.4 The assumption that fails

Theorem 5.9 requires, for each unit, a perturbation it induces — its *admissible response*. Nothing so far fixes which one.

**Assumption 5.12** (Response independence). *The four-column verdict is independent of the choice of admissible response, within threshold  $\theta$ .*

**Proposition 5.13.** *If Theorem 5.12 fails then the verdict depends on an arbitrary choice made by the analyst, and two analysts running the same comparison may correctly obtain different verdicts.*

*Proof.* Immediate: the verdict is a function of  $D^r$ , which is a function of the chosen response.  $\square$

We report in Section 13 that Theorem 5.12 *fails empirically* on solved biochemical networks. We do not repair it here. We instead require the language to make the choice visible, which is the most a language can honestly do about an open problem: see Theorem 6.7 and Section 13.

## 6 Design: eleven rules

Every rule below is derived from a result above. We state the rule, the result forcing it, and the consequence for a user.

**Rule 6.1** (Sequences are opaque). The type **seq** supports no indexing, no slicing, no base access, no concatenation and no equality. A **seq** may only be consumed by a projection into **profile**, **coord** or **net**.

*Forced by* Theorem 2.8 and Theorem 1.1: a sequence is a labelling, and labels carry none of the invariant. *Consequence:* a user cannot write a character-level comparison, because there is no expression denoting a character. This is discussed at length in Section 6.1 because it is the rule users find hardest.

**Rule 6.2** (Every value carries residue). Every value of a comparison type carries a pair  $(\beta, \varrho)$  with  $\varrho \geq \beta > 0$ . Residue is not optional and not erasable; it may be consumed by **bind** or explicitly abandoned by **drop**, which the report records.

*Forced by* Theorem 2.4. *Consequence:* a program that throws information away must say so in its own text.

**Rule 6.3** (No zero-residue constructor). There is no expression in synopsis whose type ascribes  $\varrho = 0$ .

*Forced by* Theorem 2.4: zero is outside the range of the quantity. *Consequence:* **identical** and **percent\_identity == 100** are not writable (Theorem 9.5).

**Rule 6.4** (Verdicts are four-ary). The verdict constructor **align** takes exactly a central pair and a response pair. Supplying only the central pair is an arity error at parse time.

*Forced by* Theorem 5.7 and Theorem 5.10. *Consequence:* the commonest scientific error in this domain — concluding functional sameness from content sameness — is a *syntax* error.

**Rule 6.5** (Frames are lexical). Every value is created inside an **under** block naming a receiver and an encoding. Values from different blocks have different types and cannot be compared.

*Forced by* Theorem 3.5 and Theorem 3.6. *Consequence:* comparing a nucleotide embedding to a protein embedding, or a value from receiver  $N$  to one from  $N'$ , fails to typecheck (Theorem 9.7).

**Rule 6.6** (No default thresholds). Every numeric parameter that can change a verdict must appear in the program text. There are no defaults.

*Forced by* nothing in the mathematics — this one is methodological [54, 21]. *Consequence:* the report can print the full parameter set because the parser has seen all of it.

**Rule 6.7** (Named responses). The response constructor is **response  $u$  by  $\pi$** , where  $\pi$  must be a named, in-scope perturbation map. There is no anonymous or inferred response.

*Forced by* Theorem 5.13 and the negative result of Section 13. *Consequence:* every program records which arbitrary choice it made, so the open problem is visible in the source and in the report rather than buried in a library default.



**Rule 6.8** (Bounded iteration only). The only iteration forms are **sweep** over an arithmetic range or a literal list, **for** over the finite item set of a structure, and **relax until quiescent**. There is no **while**, no recursion, and no user-defined fixpoint.

*Forced by* the desire for Theorem 9.1, made affordable by Theorem 4.10. *Consequence:* every synopsis program halts, and the compiler can state the bound.

**Rule 6.9** (Relaxation must be floored). **relax ... until quiescent** typechecks only when a step floor  $\eta > 0$  is supplied as a literal or an expression provably positive. The compiler then emits  $\lceil D_0/\eta \rceil$  into the report.

*Forced by* Theorem 4.7 and Theorem 4.10. *Consequence:* the termination proof of each program is a by-product of compiling it.

**Rule 6.10** (Three result types). Comparison methods produce distinct types according to their index set: **profile** (indexed by lag), **ranked** (indexed by database entry), **verdict** (an element of a three-valued set over a structure). These do not unify and are not coercible.

*Forced by* the fact that the index sets genuinely differ; see Section 6.2. *Consequence:* a user cannot accidentally feed a lag-indexed correlation into a database-ranking consumer.

**Rule 6.11** (Programs emit reports). The terminal form of a synopsis program is a report: a document carrying claims, their verdicts, their residues, their parameters and their certificates. A program does not “return a number”.

*Forced by* Theorem 4.11: quiescence coexists with positive residue, so a scalar cannot express the result. *Consequence:* outputs are auditable in the sense of [49, 64, 38].

## 6.1 On Theorem 6.1, at length

Users object to opacity, so it deserves a defence beyond the citation of Theorem 2.8.

The objection is that sequences are data and data should be manipulable. The answer is that in synopsis the sequence is not the datum; the *structure it labels* is. Opening a **seq** to indexing would immediately permit writing  $s[i] == t[i]$  and, from there, percent identity — which Theorem 6.3 and Theorem 9.5 exist to forbid. A rule that can be circumvented by three lines of user code is not a rule.

The practical question is then whether opacity costs anything. We claim it does not, on the evidence of the three experiment families we specify in Section 10. Each begins by *channelising* the sequence — mapping it to a numeric matrix under a declared encoding — and never touches a base again. Motif detection consumes channels via correlation; homology search consumes them via a spectral transform; functional adjudication consumes a contact structure derived from pairing rather than from the string. None of them needs  $s[i]$ .

What opacity does cost is the ability to write *ad hoc* string surgery inside a synopsis program. We regard this as a feature: such surgery is data preparation, belongs upstream, and should be visible in the provenance record rather than hidden inside an analysis script.

## 6.2 Three families, three index sets

Theorem 6.10 deserves justification because the temptation to unify is strong and we ourselves initially yielded to it.

- **Matched filtering.** A query of length  $L_q$  is slid along a target of length  $L_t$  and a score computed at each offset. The result is indexed by lag  $k \in \{0, \dots, L_t - L_q\}$  [59, 43]. Its natural consumer is peak detection; its natural output is a set of *positions*.

- **Database search.** A query embedding is compared against  $N$  stored embeddings. The result is indexed by *database entry*  $i \in \{0, \dots, N - 1\}$  [1, 56]. Its natural consumer is top- $K$  selection; its natural output is a *ranking*.
- **Functional adjudication.** Two structures are compared. The result is not indexed at all: it is a single element of  $\{\text{CORRESPOND}, \text{DIVERGE}, \text{DECLINE}\}$  with four demand values attached.

A single **compare** returning a single type would have to pick one of these index sets and coerce the others. Coercing a lag index to a database index is meaningless; coercing either to a scalar discards exactly the information the method exists to produce. Synopsis therefore keeps one *keyword*, **compare**, and three *result types*, with the parser dispatching on the named method. The user writes the same verb; the typechecker knows which noun came back.

## 7 The synopsis language

Files carry the extension `.syp`.

### 7.1 Lexical structure

Comments run from `#` to end of line. Identifiers match `[A-Za-z_][A-Za-z0-9_]*`. Numeric literals are decimal, with an optional exponent; *there are no unsuffixed bare numbers in threshold position* — see Section 7.5. String literals are double-quoted and are used only for file paths and claim text. The language is case-sensitive and whitespace-insensitive except that newlines terminate statements.

### 7.2 Types

| Type                                                | Carries                               | Meaning                                                                 |
|-----------------------------------------------------|---------------------------------------|-------------------------------------------------------------------------|
| <code>seq</code>                                    | —                                     | An opaque sequence. No observers.                                       |
| <code>frame</code>                                  | receiver, encoding                    | A lexical scope for values.                                             |
| <code>profile<sub><math>\phi</math></sub></code>    | $(\beta, \varrho)$ , lag index        | Lag-indexed score vector in frame $\phi$ .                              |
| <code>coord<sub><math>\phi</math></sub></code>      | $(\beta, \varrho)$ , dimension        | A point in an embedding space.                                          |
| <code>ranked<sub><math>\phi</math></sub></code>     | $(\beta, \varrho)$ , entry index      | A ranking over a database.                                              |
| <code>peaks<sub><math>\phi</math></sub></code>      | $(\beta, \varrho)$ , positions        | Detected positions with scores.                                         |
| <code>net<sub><math>\phi</math></sub></code>        | $(\beta, \varrho)$ , $\Omega$         | A contact structure.                                                    |
| <code>unit<sub><math>\phi</math></sub></code>       | $(\beta, \varrho)$ , anchors, residue | A central column.                                                       |
| <code>response<sub><math>\phi</math></sub></code>   | $(\beta, \varrho)$ , named $\pi$      | A response column.                                                      |
| <code>corr<sub><math>\phi, \psi</math></sub></code> | partial map                           | A correspondence between two structures.                                |
| <code>verdict</code>                                | four demands, $\theta$ , certificate  | <code>CORRESPOND</code>   <code>DIVERGE</code>   <code>DECLINE</code> . |
| <code>real<sup>+</sup></code>                       | —                                     | A strictly positive real.                                               |
| <code>depth</code>                                  | —                                     | A non-negative partition depth (Section 7.7).                           |

The subscript  $\phi$  is a *frame index*, assigned by the enclosing **under** block. Two types with different frame indices are different types. This single device implements Theorem 6.5 and yields Theorem 9.7.

### 7.3 Grammar

We give the concrete grammar in EBNF. Nonterminals are *italic*; terminals are **typewriter**.

```

program ::= { decl } { frame_block } report_stmt

decl ::= "open" ident "=" string
      | "let" ident "=" expr
      | "method" ident "=" method_spec
      | "require" predicate

frame_block ::= "under" ident "{" { stmt } "}"

stmt ::= binding | claim_stmt | sweep_stmt | for_stmt
      | relax_stmt | record_stmt | drop_stmt

binding ::= "let" ident [":" type] "=" expr
          | "bind" ident "," ident "=" expr # value, residue

expr ::= project_expr | compare_expr | detect_expr
      | align_expr | net_expr | prim_expr

project_expr ::= "project" expr "by" projector
projector ::= "channels" "(" enc ")"
            | "spectral" "(" "coeffs" "=" int ")"
            | "contact" "(" "medium" "=" num ")"
            | "cardinal" "(" ")"
enc ::= "dna" | "protein"

compare_expr ::= "compare" expr "against" expr "by" method_ref

detect_expr ::= "detect" detect_kind "in" expr "{" params "}"
detect_kind ::= "peaks" | "top"

align_expr ::= "align" "central" "(" expr "," expr ")"
              "response" "(" expr "," expr ")"
              "under" corr_ref "{" params "}"

net_expr ::= "perturb" expr "by" ident
            | "join" expr "and" expr
            | "unit" expr "anchors" int
            | "response" expr "by" ident

relax_stmt ::= "relax" ident "until" "quiescent"
              "{" "eta" "=" num ";" "theta" "=" num "}"

sweep_stmt ::= "sweep" ident "in" range "{" { stmt } "}"
range ::= num ".." num "step" num | "[" num { "," num } "]"

for_stmt ::= "for" ident "in" "items" "(" expr ")"
           [ "where" predicate ] "{" { stmt } "}"

claim_stmt ::= "claim" string "=" expr
record_stmt ::= "record" ident { "," ident }
drop_stmt ::= "drop" ident
report_stmt ::= "report" "to" string

params ::= param { ";" param }
param ::= ident "=" num
method_spec ::= "xcorr" "(" "normalised" ")"
              | "shader" "(" "cosine" ")"
              | "smith_waterman" "(" params ")"
              | "jaccard" "(" "k" "=" int ")"

```

## 7.4 Frames and the under block

```

open wildtype = "wt.fa"
open variant = "var.fa"

under nucleotide_frame {
  let a = project wildtype by channels(dna)
  let b = project variant by channels(dna)
  let r = compare a against b by xcorr(normalised)
}

under protein_frame {
  let p = project wildtype by channels(protein)
  # let bad = compare a against p by xcorr(normalised)
  # ~ scope error: 'a' was created in nucleotide_frame
}

```

The commented line does not fail at runtime with a shape mismatch; it fails at typecheck, because `profilenucleotide_frame` and `profileprotein_frame` are distinct types. This is Theorem 9.7.

## 7.5 Units and residue

Every comparison value is a pair. The user may bind both:

```

under N {
  bind score, res = compare q against t by xcorr(normalised)
  # 'res' is in scope and MUST be consumed:
  record res
}

```

Leaving `res` unconsumed is a compile error. The user may abandon it deliberately:

```

drop res # recorded in the report as an explicit abandonment

```

which typechecks, and prints in the report as a line reading `residue abandoned at line  $n$` . There is no silent path.

## 7.6 Relaxation

```

under N {
  let d = compare unitA against unitB by demand
  relax d until quiescent { eta = 0.005 ; theta = 0.01 }
}

```

The compiler checks that `eta` is a positive literal or a provably-positive expression. If it is not, the program is rejected with

```

error: 'relax' requires a step floor bounded below (Assumption: effective update).
eta = 0.0 is not positive; no termination bound can be emitted.

```

If it is, the compiler emits into the report the discharged obligation

$$\text{steps} \leq \lceil D_0/\eta \rceil = \lceil 0.4130/0.005 \rceil = 83,$$

computed from the value of  $D_0$  available at that point. The termination of the loop is thus a theorem the compiler states, not a hope the user holds.

## 7.7 Partition depth

Some analyses need an additive measure of how finely a space has been subdivided — how many nested distinctions have been drawn. We include one primitive for it.

**Definition 7.1** (Partition depth). If a space is subdivided successively into  $k_1, k_2, \dots, k_m$  parts, its *partition depth* at base  $b$  is  $\text{depth} = \sum_{j=1}^m \log_b k_j$ .

**Proposition 7.2** (Additivity). *If two subdivisions are independent, the depth of their product is the sum of their depths.*

*Proof.* The product subdivision has parts  $k_j k'_l$  over the product index, and  $\log_b(k_j k'_l) = \log_b k_j + \log_b k'_l$ ; summing over the product index and factoring gives the claim.  $\square$

Depth is a **depth**-typed value and cannot be added to a  $\text{real}^+$ . Its use is to state how deep a hierarchical address is before it is used as a prefilter (Section 10.2).

## 7.8 The typing judgement

We write  $\Gamma; \phi \vdash e : \tau$  for “in environment  $\Gamma$  and frame  $\phi$ , expression  $e$  has type  $\tau$ ”. Selected rules:

|            |                                                                                                                                                                                                                                                                                                                                                                                         |
|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (OPEN)     | $\frac{}{\Gamma; \phi \vdash \mathbf{open} \ x = s : \mathbf{seq}}$                                                                                                                                                                                                                                                                                                                     |
| (PROJ-CH)  | $\frac{\Gamma; \phi \vdash e : \mathbf{seq}}{\Gamma; \phi \vdash \mathbf{project} \ e \ \mathbf{by} \ \mathbf{channels}(c) : \mathbf{profile}_\phi(\beta, \varrho), \ \varrho \geq \beta > 0}$                                                                                                                                                                                          |
| (CMP-LAG)  | $\frac{\Gamma; \phi \vdash e_1 : \mathbf{profile}_\phi \quad \Gamma; \phi \vdash e_2 : \mathbf{profile}_\phi}{\Gamma; \phi \vdash \mathbf{compare} \ e_1 \ \mathbf{against} \ e_2 \ \mathbf{by} \ \mathbf{xcorr} : \mathbf{profile}_\phi}$                                                                                                                                              |
| (CMP-RANK) | $\frac{\Gamma; \phi \vdash e_1 : \mathbf{coord}_\phi \quad \Gamma; \phi \vdash e_2 : \mathbf{coord}_\phi^N}{\Gamma; \phi \vdash \mathbf{compare} \ e_1 \ \mathbf{against} \ e_2 \ \mathbf{by} \ \mathbf{shader} : \mathbf{ranked}_\phi}$                                                                                                                                                |
| (ALIGN)    | $\frac{\begin{array}{c} \Gamma; \phi \vdash c_A, c_B : \mathbf{unit}_\phi \quad \Gamma; \phi \vdash r_A, r_B : \mathbf{response}_\phi \\ \Gamma; \phi \vdash \varphi^c, \varphi^r : \mathbf{corr}_{\phi, \phi} \quad \theta > 0 \text{ literal} \end{array}}{\Gamma; \phi \vdash \mathbf{align} \ \mathbf{central}(c_A, c_B) \ \mathbf{response}(r_A, r_B) \ \dots : \mathbf{verdict}}$ |
| (RELAX)    | $\frac{\Gamma; \phi \vdash d : \mathbf{real}^+ \quad \eta > 0 \text{ provable}}{\Gamma; \phi \vdash \mathbf{relax} \ d \ \mathbf{until} \ \mathbf{quiescent} \ \dots : \mathbf{verdict} \ [\text{emit} \ \lceil d/\eta \rceil]}$                                                                                                                                                        |

Note what has *no* rule: there is no rule producing  $\varrho = 0$ ; there is no rule deriving **verdict** from fewer than four columns; there is no rule relating  $\mathbf{profile}_\phi$  to  $\mathbf{profile}_\psi$  for  $\phi \neq \psi$ ; and there is no elimination rule for **seq** other than projection. The refusals of Section 9 are proved by inspecting the absence of these rules.

## 8 Operational semantics

We give a small-step semantics [45] over configurations  $\langle \Sigma, \mathcal{R}, s \rangle$  where  $\Sigma$  is a store,  $\mathcal{R}$  is the accumulating report, and  $s$  is the statement sequence.

### 8.1 Projection

Channelisation is the single primitive shared by all families.

**Definition 8.1** (Channelisation). Given a sequence  $s$  of length  $L$  over an alphabet  $\mathcal{A}$  and an encoding  $c$ ,  $\mathbf{channels}(c)$  produces a matrix  $X \in \mathbb{R}^{C \times L}$ :

- $c = \mathbf{dna}$ :  $C = 4$ ,  $X_{\alpha, n} = \mathbb{K}[s_n = \alpha]$  for  $\alpha \in \{\mathbf{A}, \mathbf{C}, \mathbf{G}, \mathbf{T}\}$ , then each row is centred by subtracting its mean.

- $c = \text{protein}$ :  $C = 3$ , the rows being hydropathy [35], residue volume [65] and net charge at neutral pH, each normalised to  $[0, 1]$  and then centred.
- $c = \text{cardinal}$ :  $C = 2$ , mapping each base to a unit vector in a fixed cardinal frame; the complementary strand maps to the negated trajectory, which is the formal expression of Section 1.3.

Centring is not cosmetic: it removes the composition-only component so that subsequent correlations respond to arrangement rather than base frequency.

$$(\text{E-PROJ}) \quad \langle \Sigma, \mathcal{R}, \text{let } x = \text{project } s \text{ by channels}(c) \rangle \longrightarrow \langle \Sigma[x \mapsto X_c(s)], \mathcal{R} \cup \{\text{proj}\}, \cdot \rangle$$

## 8.2 The three comparison methods

**Definition 8.2** (Normalised cross-correlation). For query channels  $Q \in \mathbb{R}^{C \times L_q}$  and target channels  $T \in \mathbb{R}^{C \times L_t}$  with  $L_q \leq L_t$ , the score at lag  $k$  is

$$r[k] = \frac{\sum_{\gamma=1}^C \sum_{n=0}^{L_q-1} Q_{\gamma,n} T_{\gamma,n+k}}{\|Q\|_F \cdot \|T_{[k, k+L_q]}\|_F}, \quad k = 0, \dots, L_t - L_q,$$

with the convention that a zero denominator yields  $r[k] = 0$ . The numerator is computed by FFT in  $O((L_q + L_t) \log(L_q + L_t))$  time [10]; the denominator uses a rolling window norm. Channels are summed *coherently*, so phase agreement across channels reinforces and disagreement cancels — discarding phase and correlating magnitudes alone is a strictly weaker procedure.  $r[k] \in [-1, 1]$  by Cauchy–Schwarz.

**Definition 8.3** (Spectral embedding). For channels  $X \in \mathbb{R}^{C \times L}$  and a coefficient count  $m$ , let  $\hat{X}_\gamma$  be the real FFT of row  $\gamma$ . The embedding is

$$E = \frac{1}{\max(L, 1)} \left( |\hat{X}_{\gamma,j}| \right)_{\gamma \in [C], j \in \{1, \dots, m\}} \in \mathbb{R}^{Cm},$$

zero-padded if  $L$  is short. The DC term  $j = 0$  is *skipped*: it encodes only mean composition, which centring has already removed and which would otherwise dominate the norm. Division by  $L$  makes embeddings of different-length sequences comparable. Embeddings are then  $\ell_2$ -normalised, so cosine similarity is a dot product.

**Definition 8.4** (Shader distance and hierarchical address). For an  $\ell_2$ -normalised query  $q$  and a normalised database matrix  $B \in \mathbb{R}^{N \times Cm}$ , the distance vector is  $d = 1 - Bq \in [0, 2]^N$ . A *hierarchical address* of depth  $D$  for a point in  $[0, 1]^3$  is obtained by axis-cyclic bisection: at step  $j$  bisect along axis  $j \bmod 3$  and emit the bit. Two points sharing an address prefix of length  $D$  lie in a common cell, giving a prefilter in the spirit of [39, 7]; candidates surviving the prefilter are reranked exactly by Smith–Waterman [55].

**Definition 8.5** (Contact projection). `contact(medium =  $w$ )` maps a solved network — a set of species with pairwise contact costs — to a contact structure by taking the costs as edge weights and attaching the medium to every species at weight  $w$ . Setting  $w$  to the minimum observed cost makes the medium the cheapest distinction available and realises the floor without inventing a scale; any other choice must be written explicitly, and the report prints it.

### 8.3 Verdict evaluation

|             |                                                                                                                                                            |  |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| (E-ALIGN)   | $D^c = D_{A \rightarrow B} + D_{B \rightarrow A} \quad D^r = D_{R_A \rightarrow R_B} + D_{R_B \rightarrow R_A}$                                            |  |
|             | no relaxation declined                                                                                                                                     |  |
| (E-DECLINE) | $\text{align} \dots \rightarrow \begin{cases} \text{CORRESPOND} & D^c \leq \theta \wedge D^r \leq \theta \\ \text{DIVERGE} & \text{otherwise} \end{cases}$ |  |
|             | some relaxation reached an undefined update                                                                                                                |  |
|             | $\text{align} \dots \rightarrow \text{DECLINE}$                                                                                                            |  |

### 8.4 Cost model

The language commits to complexity bounds so that a user can predict cost from the program text.

| Operation             | Cost                                | Note                      |
|-----------------------|-------------------------------------|---------------------------|
| <b>channels</b>       | $O(CL)$                             | linear scan               |
| <b>xcorr</b>          | $O((L_q + L_t) \log(L_q + L_t))$    | FFT [10]                  |
| <b>spectral</b>       | $O(CL \log L)$                      | per sequence              |
| <b>shader</b>         | $O(NCm)$                            | one matrix–vector product |
| prefix prefilter      | $O(N)$ then $O( \text{cand} )$      | depth- $D$ address        |
| <b>smith_waterman</b> | $O(L_a L_b)$ per candidate          | exact rerank              |
| $\sigma(u \mid v)$    | polynomial                          | max-flow [17, 57]         |
| all-pairs $\sigma$    | $ V  - 1$ flows                     | Gomory–Hu [22]            |
| <b>relax</b>          | $\leq \lceil D_0/\eta \rceil$ steps | Theorem 4.10              |

*Remark 8.6* (Exact cuts in validation). A validation suite should compute minimum cuts by exhaustive enumeration over separating subsets, not by an approximate max-flow routine, so that agreement between a computed value and a predicted one cannot be an artefact of two approximations meeting. This is exponential and is affordable only on small structures; synopsis therefore admits an **exact** annotation on **contact** that selects enumeration and refuses structures above a stated size.

## 9 Metatheory

We now prove the properties that justify the design. Throughout, a *well-typed program* is one derivable in the judgement of Section 7.

### 9.1 Totality

**Theorem 9.1** (Strong normalisation). *Every well-typed synopsis program terminates.*

*Proof.* We exhibit a well-founded measure. Assign to each statement sequence the lexicographic pair  $(\mathcal{I}, \mathcal{S})$  where  $\mathcal{S}$  is the number of statements remaining and  $\mathcal{I}$  is the sum, over all enclosing iteration constructs, of the remaining iteration count.

Every construct that can repeat has a count fixed before it begins:

- **sweep**  $x$  **in**  $a..b$  **step**  $s$  has  $\lfloor (b - a)/s \rfloor + 1$  iterations, determined by three values evaluated on entry; the grammar admits no expression for  $s$  that is not positive.
- **sweep**  $x$  **in**  $[\cdot]$  has the literal list length.

- **for**  $u$  **in**  $\text{items}(e)$  ranges over  $I = V \setminus \{\mathbf{m}\}$ , finite by Theorem 2.1; the optional **where** guard only removes elements.
- **relax** has at most  $\lceil D_0/\eta \rceil$  iterations by Theorem 4.10, and the typing rule (RELAX) admits the construct only when  $\eta > 0$  is established, so the bound is a positive integer computed on entry.

There is no other repeating construct: the grammar has no **while**, no user-defined function and hence no recursion, and no fixpoint operator. Each step of evaluation strictly decreases  $(\mathcal{I}, \mathcal{S})$  in the lexicographic order on  $\mathbb{N}^2$ , which is well-founded. Hence no infinite reduction sequence exists.  $\square$

**Corollary 9.2** (Cost is predictable). *For each program the compiler can compute a closed-form upper bound on the number of primitive operations, since every iteration count is available statically or at loop entry.*

## 9.2 Floor preservation

**Theorem 9.3** (Floor preservation). *Let  $e$  be a well-typed expression of a comparison type evaluating to a value carrying  $(\beta, \varrho)$ . Then  $\varrho \geq \beta > 0$ .*

*Proof.* Induction on the typing derivation. The base cases are the projection rules, whose conclusions carry  $\varrho \geq \beta > 0$  because Theorem 8.5 produces a structure with strictly positive weights (Theorem 2.1) and Theorem 2.4 then applies.

For the inductive step we check each rule that combines values. (CMP-LAG) and (CMP-RANK) produce residues that are sums or minima of subterm residues, each  $\geq \beta > 0$  by hypothesis, and a finite sum or minimum of quantities  $\geq \beta$  is  $\geq \beta$ . **perturb** is non-negative by Theorem 2.15, so by Theorem 2.16 it does not decrease any separation cost and does not lower  $\beta$  unless it acts on the minimising edge, in which case it raises it. **join** forms a disjoint union sharing the medium; the floor of the union is the minimum of the two floors, still strictly positive, and the typing rule for **join** requires both operands to be in the same frame so that the medium is shared. No rule subtracts residues. Hence the invariant is preserved.  $\square$

**Theorem 9.4** (No zero residue). *There is no well-typed synopsis expression whose value carries  $\varrho = 0$ .*

*Proof.* Immediate from Theorem 9.3:  $\varrho \geq \beta > 0$ .  $\square$

## 9.3 The refusal theorems

These state that certain scientifically invalid claims are not expressible. Each is proved by exhibiting the absence of a typing rule, which is a statement about the fixed grammar of Section 7 and so is a genuine theorem about the language rather than an observation about a particular implementation.

**Theorem 9.5** (Exactness is inexpressible). *No well-typed synopsis program denotes the proposition “ $u$  and  $v$  are identical”, in the sense that no expression evaluates to a value asserting zero separation cost between two items.*

*Proof.* Such an expression would have to produce either (a) a comparison value with  $\varrho = 0$ , excluded by Theorem 9.4; or (b) a boolean equality on **seq**, for which no rule exists — the only elimination form for **seq** is (PROJ-CH) and its variants, all of which produce comparison types carrying residue; or (c) a numeric expression reconstructing identity from base-level access, which requires an indexing operator on **seq** that the grammar does not contain (Theorem 6.1). All three routes are closed.  $\square$



**Theorem 9.6** (Arity of verdicts). *No well-typed synopsis program derives a value of type verdict from fewer than two central and two response columns.*

*Proof.* (ALIGN) is the only rule with **verdict** in its conclusion apart from (RELAX), which itself requires an **align** expression as its subject. (ALIGN) has four column premises, and a derivation lacking any premise is not a derivation. The grammar production *align\_expr* likewise requires both a **central** and a **response** clause, so an attempt to omit one fails at parse time with an arity error rather than producing a weaker but well-formed value.  $\square$

**Theorem 9.7** (Scope safety). *No well-typed synopsis program compares two values created in distinct **under** blocks.*

*Proof.* Each **under** block introduces a fresh frame index  $\phi$ , and every value constructed inside it receives type  $\tau_\phi$ . The comparison rules (CMP-LAG), (CMP-RANK) and (ALIGN) require their premises to carry the *same* index, and there is no subsumption, coercion or subtyping rule relating  $\tau_\phi$  and  $\tau_\psi$  for  $\phi \neq \psi$ . Hence no derivation exists.  $\square$

**Corollary 9.8** (Encoding safety). *A program cannot compare a nucleotide-channelised value with a protein-channelised value, nor a value registered in one receiver with one registered in another. By Theorem 3.6 the latter would be a category error rather than a tolerable approximation.*

**Theorem 9.9** (Parameter completeness). *For every well-typed program, the set of numeric parameters capable of changing any verdict appears in the program text and is recoverable by the compiler.*

*Proof.* By Theorem 6.6 the grammar productions for *detect\_expr*, *align\_expr*, *relax\_stmt* and *method\_spec* each require a *params* block, and *param* requires a literal on the right-hand side. No production permits omission. Hence every such parameter is a token in the source, and the parser collects them.  $\square$

**Corollary 9.10** (Residue accountability). *Every residue produced by a well-typed program is either bound and consumed, or explicitly dropped with a recorded source location. There is no silent discard.*

*Proof.* The **bind** form introduces both components into scope; the well-formedness condition on block exit requires every bound identifier of residue type to be consumed by **record** or **drop**. **drop** emits a report entry. A block leaving a residue identifier unconsumed does not typecheck.  $\square$

## 9.4 What the metatheory does not give

Honesty requires stating the boundary.

Totality (Theorem 9.1) says programs halt; it says nothing about whether they halt *soon*. A **sweep** over  $10^9$  points is well-typed. The cost model of Section 8.4 is the tool for that, not the type system.

Floor preservation (Theorem 9.3) says residues stay positive; it does not say they are *small*, and a large residue is a scientific problem the language reports but does not solve.

The refusal theorems say certain claims cannot be *written*; they do not say the claims a user does write are *true*. Synopsis rules out a family of category errors. It cannot rule out a badly chosen threshold, a mis-specified receiver, or — most importantly — the open problem of Section 13.

## 10 Worked programs

We give one complete program per experiment family. Each is a full **.syp** file; none is a fragment.

## 10.1 Matched-filter motif detection

*Question.* Where in a target does a known motif occur, and with what confidence against a background?

```
1  # Locate a motif in a target by coherent multichannel matched filtering.
2  # All thresholds are explicit; none has a default.
3
4  open motif = "motif.fa"
5  open target = "chr7_region.fa"
6
7  under nucleotide {
8
9      let q = project motif by channels(dna)
10     let t = project target by channels(dna)
11
12     # Lag-indexed: the result type is profile, not ranked.
13     bind r, res_r = compare q against t by xcorr(normalised)
14
15     # Peak calling. Every parameter is stated.
16     let hits = detect peaks in r {
17         z = 4.0 ; # z-score against local background
18         min_distance = 30 ; # greedy maximum suppression radius
19         min_score = 0.35
20     }
21
22     claim "motif occurs at least once above z=4" = hits
23     record hits, res_r
24 }
25
26 report to "motif_scan.report"
```

Listing 1: motif\_scan.syp

Points of note. The residue `res_r` is bound and recorded; had it been ignored the program would not compile (Theorem 9.10). `r` has type `profilenucleotide` and could not be passed to a top- $K$  selector, which consumes `ranked` (Theorem 6.10). The three peak parameters are in the source, so the report can print them (Theorem 9.9).

A sweep over the decision threshold, to show the verdict's sensitivity rather than assert one value:

```
sweep z0 in 2.0..6.0 step 0.5 {
    let h = detect peaks in r { z = z0 ; min_distance = 30 ; min_score = 0.35 }
    record h
}
```

This terminates in exactly 9 iterations, fixed on entry (Theorem 9.1).

## 10.2 Spectral homology search

*Question.* Which entries of a database are plausible homologues of a query, when sequence-level search is too slow or too insensitive?

```
1  # Two-stage search: cheap spectral prefilter, exact rerank.
2
3  open query = "query.fa"
4  open db = "swissprot_subset.fa"
5
6  under residue_space {
7
```

```

8   let e = project query by spectral(coeffs = 8)
9   let DB = project db by spectral(coeffs = 8)
10
11  # Entry-indexed: the result type is ranked.
12  bind s, res_s = compare e against DB by shader(cosine)
13
14  let cand_s = detect top in s {
15    k = 200 ; # prefilter width
16    depth = 12 # hierarchical address prefix length
17  }
18
19  # Exact rerank of survivors only. Costs  $O(L_a * L_b)$  per candidate.
20  bind final, res_f = compare e against cand_s by smith_waterman(
21    match = 2 ; mismatch = -1 ; gap = -2
22  )
23
24  claim "top 20 by exact rerank" = final
25  record final, res_s, res_f
26 }
27
28 report to "homology.report"

```

Listing 2: homology.syp

The two-stage shape is forced by the cost model: the shader stage is one matrix–vector product,  $O(NCm)$ , and the exact stage is quadratic per candidate, so the address depth is the parameter trading sensitivity for work. It is written in the source rather than defaulted. Note also that `coeffs = 8` appears twice and must match — comparing embeddings of different dimension is a type error, since the dimension is part of  $\text{coord}_\phi$ .

### 10.3 Four-column functional adjudication

*Question.* Do a wild-type unit and a variant unit do the same thing, given that content comparison cannot answer this (Theorem 5.1)?

```

1  # The central question: same function, not same sequence.
2
3  open wt = "wildtype.solved"
4  open var = "variant.solved"
5
6  # Named perturbation maps. Rule: responses may not be anonymous,
7  # because the verdict's independence of this choice is OPEN.
8  method pi_edge_raise = perturb_incident(magnitude = 0.10)
9
10 under cell_receiver {
11
12   let Nwt = project wt by contact(medium = 0.05)
13   let Nvar = project var by contact(medium = 0.05)
14
15   # Central columns: the units with their anchor/residue split.
16   let cA = unit Nwt anchors 3
17   let cB = unit Nvar anchors 3
18
19   # Response columns: what each unit does to the receiver.
20   # 'by pi_edge_raise' is mandatory -- the arbitrary choice is named.
21   let rA = response cA by pi_edge_raise
22   let rB = response cB by pi_edge_raise
23

```

```

24 let phi_c = corr from cA to cB by species_name
25 let phi_r = corr from rA to rB by species_name
26
27 # FOUR columns. Dropping 'response(...)' is an arity error.
28 let v = align central(cA, cB) response(rA, rB)
29       under phi_c, phi_r { theta = 0.01 }
30
31 relax v until quiescent { eta = 0.005 ; theta = 0.01 }
32
33 claim "variant is functionally equivalent to wild type" = v
34 record v
35 }
36
37 report to "adjudicate.report"

```

Listing 3: adjudicate.syp

Three things this program cannot do, by construction:

- (i) It cannot conclude equivalence from `cA` and `cB` alone — deleting the `response` clause is an arity error (Theorem 9.6), so the mistake of Theorem 5.1 cannot be made.
- (ii) It cannot hide which response was chosen — `pi_edge_raise` is named and printed (Theorem 6.7).
- (iii) It cannot loop forever — `eta` is positive, so the compiler emits the bound (Theorem 4.10).

## 10.4 An annotation transfer

The stencil use of the framework: given an annotated reference and an unannotated subject, transfer annotations by correspondence rather than by sequence identity.

```

1 open ref = "reference.solved"
2 open subject = "subject.solved"
3
4 method pi_uniform = perturb_incident(magnitude = 0.10)
5
6 under tissue_receiver {
7
8   let Nr = project ref by contact(medium = 0.05)
9   let Ns = project subject by contact(medium = 0.05)
10
11   # For every item of the reference, ask whether the subject has a
12   # unit that corresponds under BOTH column pairs.
13   for u in items(Nr) where separation(u) > 0.10 {
14
15     let cA = unit u anchors 3
16     let cB = nearest_unit Ns to cA
17     let rA = response cA by pi_uniform
18     let rB = response cB by pi_uniform
19
20     let phi_c = corr from cA to cB by anchor_match
21     let phi_r = corr from rA to rB by anchor_match
22
23     let v = align central(cA, cB) response(rA, rB)
24           under phi_c, phi_r { theta = 0.02 }
25
26     claim "annotation of u transfers to its correspondent" = v
27     record v

```

```

28     }
29 }
30
31 report to "transfer.report"

```

Listing 4: `transfer.syp`

The **for** loop ranges over a finite item set with a guard, so it terminates (Theorem 9.1). The guard  $\sigma(u) > 0.10$  excludes items too close to the medium to individuate reliably — a filter the theory supplies (Theorem 2.4) rather than a heuristic. Each iteration produces a **verdict**, and a transfer that comes back **DECLINE** is recorded as such rather than silently omitted.

## 11 The report

The terminal form of every program is **report to** a path. The report is a structured document, not a log, and its schema is fixed.

| Section      | Content                                                                                                            |
|--------------|--------------------------------------------------------------------------------------------------------------------|
| PROVENANCE   | Source file hash, every <b>open</b> path and its hash, language version.                                           |
| FRAMES       | Each <b>under</b> block, its receiver and encoding, and which values were created in it.                           |
| PARAMETERS   | Every numeric parameter in the source, with its line number. Complete by Theorem 9.9.                              |
| CLAIMS       | Each <b>claim</b> string with its verdict, the four demands, and $\theta$ .                                        |
| RESIDUES     | Every residue: recorded value, or an explicit abandonment with line number (Theorem 9.10).                         |
| CERTIFICATES | For each <b>relax</b> : $D_0$ , $\eta$ , the emitted bound $\lceil D_0/\eta \rceil$ , and the realised step count. |
| DECLINES     | Every <b>DECLINE</b> , with the step at which the update became undefined.                                         |
| ASSUMPTIONS  | Each <b>require</b> assertion, verbatim, marked <i>asserted by user, not verified</i> .                            |

Two features deserve comment.

**Declines are reported, not suppressed.** A run in which half the comparisons decline is a legitimate scientific outcome — it says the comparison as posed does not decide — and a format that discards them would misrepresent the run as more conclusive than it was.

**Assumptions are marked unverified.** **require** lets a user assert, for instance, that an expansion is floor-stabilising (Theorem 2.12). The language cannot check this: it is a claim about structures not yet observed. It therefore records the assertion and labels it, so a reader of the report knows exactly which load the conclusion is placing on the author’s judgement rather than on the computation.

## 12 Validation

Every claim in this paper that can be checked by computation has been. This section states what was checked, how, and — because this is the part that carries information — what the checking changed.

## 12.1 What is checkable, and what is not

The claims fall into three kinds, and conflating them would make the exercise worthless.

**Mathematical** (Section 2–Section 5). These are statements about contact structures and are checkable numerically on explicit instances. Minimum cuts are computed by exhaustive enumeration over separating subsets rather than by max-flow: the cut is *defined* as a minimum over separating sets (Theorem 2.3), and an approximate solver agreeing with an approximate expectation could conceal a real disagreement. This is exponential, and affordable only on the small structures the validation uses — which is the point of using them.

**Language** (Section 7–Section 9). The refusal theorems say certain programs *do not typecheck*. A checker that rejected everything would satisfy every refusal theorem vacuously, so each negative program is paired with a positive one, and the four programs of Section 10 are required to typecheck *verbatim as printed*. A refusal counts only if it carries the expected error class: rejecting a cross-frame reference as a parse error would not be Theorem 9.7.

**Measurement** (Section 13). Theorem 5.12 is reported, not asserted. Recording it as an assertion would either bake in a statement we believe to be false or hide the paper’s principal finding.

## 12.2 Result

| Quantity                    | Value    |
|-----------------------------|----------|
| Claim categories            | 32       |
| of which assertions         | 31       |
| of which measurements       | 1        |
| Individual checks           | 5622     |
| Checks passed               | 5622     |
| Assertion categories failed | 0        |
| Random seed                 | 20260816 |

Of the language checks, the four programs of Section 10 typecheck as printed, and 16 negative programs are each rejected with the specific error class the corresponding theorem names — arity, scope, residue, parameter and termination violations among them.

## 12.3 What the checking changed

A validation that confirms everything has usually tested nothing. Three proofs in this paper were wrong when first written, and the corrections are recorded in place rather than quietly absorbed.

- Theorem 5.1: the original proof perturbed *one* pendant edge. That leaves the minimum cut pinned by the other pendant — the minimiser simply places the medium on the raised side — and the response is identically zero for every  $c$ . The theorem is true; the witness was not. See Theorem 5.2.
- Theorem 5.4: the original proof indexed the family by the intra-cluster weight  $W$  and asserted that  $D^{\text{central}}$  increases in  $W$ . It decreases: the cut is constant while  $\Omega$  grows. The family must be indexed by the bridge, which is the edge the cut crosses. See Theorem 5.5.
- Theorem 3.1: an earlier version claimed a light edge between non-pendant items places them at the floor. Exact enumeration gave 1.4 where the argument predicted 0.2. This is now recorded as Theorem 3.2, and it is why Theorem 5.1 insists on pendants.

A fourth correction was to a check rather than a proof. We had demanded that receiver disagreement occur in at least 20% of cases; no theorem says that, and Theorem 3.7 reports what is actually true — a minority rate, with discrepancies that are never marginal when they occur. A threshold that encodes no claim is a threshold that will eventually be tuned until it passes.

## 13 The open problem, and what the language does about it

### 13.1 The negative result

We tested Theorem 5.12 directly. Four solved steady-state biochemical networks — glycolysis, the citric-acid cycle, oxidative phosphorylation and an EGFR/MAPK cascade, together supplying 35 species and 35 contact edges — were converted to contact structures by Theorem 8.5. For every species with at least two incident non-medium edges, two admissible responses were constructed by raising one or the other incident edge by the same magnitude, and the resulting cross-demands compared. If Theorem 5.12 held, the two demands would agree within  $\theta$ .

They do not.

| Quantity                                       | Value       |
|------------------------------------------------|-------------|
| Comparisons within $\theta = 0.01$             | 28.6% of 35 |
| Mean discrepancy                               | 0.070       |
| Maximum discrepancy                            | 0.307       |
| Per-network median (glycolysis)                | 0.063       |
| Per-network median (citric-acid cycle)         | 0.046       |
| Per-network median (oxidative phosphorylation) | 0.077       |
| Per-network median (EGFR/MAPK)                 | 0.100       |

The maximum exceeds  $\theta$  by more than an order of magnitude, and *every* network has a median above  $\theta$ . The failure is systematic, not driven by one pathological pathway or one outlying species. By Theorem 5.13, the four-column verdict on these networks depends on which admissible response the analyst constructed.

### 13.2 The same failure on a second corpus

The biochemical networks are few and are not independent of one another; a reader is entitled to ask whether the result is a property of those four pathways. It is not. Repeating the construction on 200 random contact structures, with no biochemical content whatever, gives 361 comparisons and the same verdict (Section 12, E32):

| Quantity                     | Networks | Random  |
|------------------------------|----------|---------|
| Comparisons                  | 35       | 361     |
| Within $\theta = 0.01$       | 28.6%    | 77.8%   |
| Mean discrepancy             | 0.070    | 0.00555 |
| Median discrepancy           | —        | 0.00451 |
| Maximum discrepancy          | 0.307    | 0.0360  |
| Assumption holds at $\theta$ | no       | no      |

The two corpora differ in severity and agree in kind. On random structures the assumption holds far more often — 77.8% against 28.6% — and the discrepancies are an order of magnitude smaller, which is what one expects when the perturbed edges are not selected for biological significance. But 77.8% is not 100%, the maximum is still more than triple  $\theta$ , and the failure

rate does not fall with sample size. Theorem 5.12 is not approximately true and awaiting a sharper estimate; it is false, and more uniformly false the more structured the network.

This also disposes of the most natural rescue. If the discrepancies were sampling error, enlarging the corpus by an order of magnitude would shrink them. It does not, and by Theorem 3.6 it cannot: each value is exactly determined by its own structure, so there is nothing for more sampling to average away.

### 13.3 What is missing

We want to be precise about the nature of the gap, because it is easy to mistake for a tuning problem.

What is missing is **a rule fixing the map from a unit to the perturbation it induces**. The theory of Section 2–Section 4 is indifferent to which perturbation a unit is taken to cause; it only requires that one be supplied. Physically, a unit does induce a definite perturbation, so the map exists; we do not have a principled construction of it from the data available. Better thresholds do not help — lowering  $\theta$  makes agreement rarer, not commoner. More data does not help — by Theorem 3.6 the discrepancy is not sampling error. What would help is a derivation of the response map from the same contact structure the central columns are drawn from, so that it is determined rather than chosen.

### 13.4 What synopsis does about it

A language cannot solve an open problem. It can refuse to let one hide.

Theorem 6.7 requires every response column to name its perturbation map. There is no anonymous response, no inferred response and no default. Consequently:

- every program’s source records which choice was made;
- the report’s PARAMETERS section prints it (Theorem 9.9);
- two runs differing only in this choice are visibly different programs, so a disagreement between them is attributable rather than mysterious;
- and the natural robustness check — sweep the response map and see whether the verdict is stable — is a three-line **sweep** that the language makes cheap to write:

```
sweep m in [0.05, 0.10, 0.20, 0.40] {
  let pi_m = perturb_incident(magnitude = m)
  let rA = response cA by pi_m
  let rB = response cB by pi_m
  let v = align_central(cA,cB) response(rA,rB)
           under phi_c, phi_r { theta = 0.01 }
  record v
}
```

A report showing the verdict flipping across this sweep is a report telling the truth about the state of the theory. We prefer that to a report showing a single confident verdict obtained by a default nobody chose.

## 14 Related work

**Sequence comparison.** The dynamic-programming tradition [40, 55] and its heuristic descendants [1, 2, 9, 56] optimise an alignment score — a content quantity. Profile methods [15, 14] and substitution matrices [24] enrich the content model without leaving it. Sketching



approaches [42, 28] trade exactness for speed, again within content. Theorem 5.7 applies to all of them: they compute one coordinate of a two-dimensional quantity. This is not a criticism of their execution; it is a statement about what they are measuring.

**Genomic signal processing.** Representing sequence as a numeric signal has a long history [61, 3], and the channelisation of Theorem 8.1 follows it. Our contribution here is not the representation but its *typing*: the frame index makes an encoding choice part of the type, so that the common error of comparing across encodings is caught statically.

**Function prediction and annotation.** Large-scale evaluations show function prediction to be substantially unsolved [47], and misannotation to propagate through databases [51]. Structure prediction [29] has transformed the structural half of the problem without settling the functional one. Ontology and pathway resources [4, 30] supply the vocabulary in which function is stated but not a criterion for deciding sameness. Our four-column criterion is such a criterion, blind to annotation by construction.

**Networks and perturbation.** Treating cellular function as a network property is standard [6], and genome-scale perturbation screens now supply the response measurements the response columns need [11, 13, 48]. The framework here is, in that light, a proposal for what to *compute* from such screens: not a similarity between expression profiles but a cross-demand between contact structures.

**Graph cuts.** Minimum cuts and their duality with flows [17, 22, 57, 31, 52] are the computational substrate. Normalised cuts in vision [53, 8] share the intuition that a boundary is the meaningful object; spectral connectivity [16] offers a relaxation we do not use because we need exactness for validation.

**Domain-specific languages.** The case for a DSL over a library [26, 37, 18] is usually made on expressiveness. Ours is made on refusal, which places it nearer to dimension-typed numerics [32], linear types [62] and refinement types [19], where the type system exists to make certain wrong programs unwritable. Totality follows [60]. Biological DSLs [44, 36] and exchange formats [25] model systems; synopsis adjudicates claims about them, which is a different job. Workflow systems [34, 12] give reproducible orchestration but are agnostic about the scientific validity of the steps they orchestrate; the report of Section 11 is closer in spirit to provenance models [38, 64].

**Reproducibility.** The no-defaults rule is a direct response to the analytic-flexibility literature [54, 21, 27, 5, 49]. Our contribution is to move the discipline from a checklist into a grammar.

## 15 Limitations

We list what we do not claim.

- L1. Response independence is open.** Section 13. This is the principal gap and it is not small: without a rule fixing the response map, the four-column verdict is analyst-dependent. Every other limitation here is minor by comparison.
- L2. Contact structures must be supplied.** Synopsis consumes solved networks; it does not solve them. Where the contact costs come from — kinetics, perturbation screens, inference — is outside the language, and a bad structure yields a confident wrong verdict.
- L3. Floor-stabilisation is asserted, not checked.** Theorem 2.12 is a hypothesis about structures not yet observed. **require** records it and marks it unverified; nothing verifies it.
- L4. The anchor count  $r$  is a free parameter.** Theorem 4.3 fixes the tie-breaking but not  $r$ . The language forces  $r$  into the source and the report, which makes it visible, not principled.

- L5. Totality bounds steps, not time.** Theorem 9.1 and the remark in Section 9. A well-typed program may still be infeasible.
- L6. No generative capability.** By the decision of Section 1.4, synopsis certifies but does not design. Users wanting sequence design must generate elsewhere and certify here.
- L7. Empirical scope is narrow.** The negative result of Section 13 rests on four networks and 35 species. It is enough to refute Theorem 5.12 — refutation needs only counterexamples — but not enough to characterise how the assumption fails.
- L8. Opacity has a cost we assert rather than measure.** Section 6.1 argues from three experiment families that Theorem 6.1 loses nothing. A wider survey could falsify this, and the honest position is that we have checked three families, not all of them.

## 16 Conclusion

The question practitioners ask of two genomic units is whether they do the same thing. The vocabulary available answers a different question — whether they are written the same way — and Theorem 5.7 shows the substitution is not an approximation but a projection that loses a dimension. Two units may be indistinguishable in content and unbounded in effect (Theorem 5.1); two may be maximally divergent in content and identical in effect (Theorem 5.4).

Occupying the missing dimension requires a theory in which identity is a cut rather than a label (Theorem 2.7, Theorem 2.8), distinction has a strictly positive price (Theorem 2.4), value is receiver-relative (Theorem 3.5), and correspondence can be certified blind (Theorem 4.4) with guaranteed termination (Theorem 4.10). It requires a verdict with four columns and three values (Theorem 5.9).

It also requires, we argue, a language rather than a library — because the results above are largely *prohibitions*, and a prohibition that lives in documentation is not enforced. Synopsis makes exactness inexpressible (Theorem 9.5), single-column verdicts an arity error (Theorem 9.6), cross-frame comparison a scope error (Theorem 9.7), silent residue discard impossible (Theorem 9.10), and every verdict-relevant parameter mandatory (Theorem 9.9); and it terminates (Theorem 9.1) with the bound printed.

The name records the ambition. The synoptic gospels are read in parallel columns not because they agree word for word but because they align sufficiently for the columns to be laid alongside, with the residue displayed rather than reconciled away. That is what genomic comparison should be doing, and Theorem 4.11 says exactly why it is coherent to do so: two structures may place no demand on each other while remaining fully distinguishable. Sufficient alignment, not exact alignment.

We end where we are weakest. Theorem 5.12 fails on real networks — 28.6% agreement, maximum discrepancy 0.307 against  $\theta = 0.01$  — and we have no rule fixing the response map. We have built the language so that this cannot be concealed: no response may be anonymous, and the sweep that exposes an unstable verdict is three lines. A language that makes its own open problem visible in every program that touches it is, we think, the correct thing to build while the problem remains open.

## References

- [1] Stephen F. Altschul, Warren Gish, Webb Miller, Eugene W. Myers, and David J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.
- [2] Stephen F. Altschul, Thomas L. Madden, Alejandro A. Schäffer, Jinghui Zhang, Zheng Zhang, Webb Miller, and David J. Lipman. Gapped BLAST and PSI-BLAST: A new

- generation of protein database search programs. *Nucleic Acids Research*, 25(17):3389–3402, 1997.
- [3] Dimitris Anastassiou. Genomic signal processing. *IEEE Signal Processing Magazine*, 18(4):8–20, 2001.
  - [4] Michael Ashburner, Catherine A. Ball, et al. Gene ontology: Tool for the unification of biology. *Nature Genetics*, 25(1):25–29, 2000.
  - [5] Monya Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533(7604):452–454, 2016.
  - [6] Albert-László Barabási and Zoltan N. Oltvai. Network biology: Understanding the cell’s functional organization. *Nature Reviews Genetics*, 5(2):101–113, 2004.
  - [7] Jon Louis Bentley. Multidimensional binary search trees used for associative searching. *Communications of the ACM*, 18(9):509–517, 1975.
  - [8] Yuri Boykov and Vladimir Kolmogorov. An experimental comparison of min-cut/max-flow algorithms for energy minimization in vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(9):1124–1137, 2004.
  - [9] Benjamin Buchfink, Chao Xie, and Daniel H. Huson. Fast and sensitive protein alignment using DIAMOND. *Nature Methods*, 12(1):59–60, 2015.
  - [10] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of Computation*, 19(90):297–301, 1965.
  - [11] Michael Costanzo, Benjamin VanderSluis, et al. A global genetic interaction network maps a wiring diagram of cellular function. *Science*, 353(6306):aaf1420, 2016.
  - [12] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, et al. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.
  - [13] Atray Dixit, Oren Parnas, Biyu Li, et al. Perturb-seq: Dissecting molecular circuits with scalable single-cell RNA profiling of pooled genetic screens. *Cell*, 167(7):1853–1866, 2016.
  - [14] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, 1998.
  - [15] Sean R. Eddy. Accelerated profile HMM searches. *PLoS Computational Biology*, 7(10):e1002195, 2011.
  - [16] Miroslav Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.
  - [17] L. R. Ford and D. R. Fulkerson. Maximal flow through a network. *Canadian Journal of Mathematics*, 8:399–404, 1956.
  - [18] Martin Fowler. *Domain-Specific Languages*. Addison-Wesley, 2010.
  - [19] Tim Freeman and Frank Pfenning. Refinement types for ML. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 268–277, 1991.
  - [20] Michael Y. Galperin and Eugene V. Koonin. Divergence and convergence in enzyme evolution. *Journal of Biological Chemistry*, 287(1):21–28, 2012.

- [21] Andrew Gelman and Eric Loken. The garden of forking paths. *Department of Statistics, Columbia University*, 2013.
- [22] R. E. Gomory and T. C. Hu. Multi-terminal network flows. *Journal of the Society for Industrial and Applied Mathematics*, 9(4):551–570, 1961.
- [23] Johann Jakob Griesbach. *Synopsis Evangeliorum Matthaei, Marci et Lucae*. Halle, 1776.
- [24] S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992.
- [25] M. Hucka, A. Finney, H. M. Sauro, et al. The systems biology markup language (SBML): A medium for representation and exchange of biochemical network models. *Bioinformatics*, 19(4):524–531, 2003.
- [26] Paul Hudak. Building domain-specific embedded languages. *ACM Computing Surveys*, 28(4es):196, 1996.
- [27] John P. A. Ioannidis. Why most published research findings are false. *PLoS Medicine*, 2(8):e124, 2005.
- [28] Paul Jaccard. The distribution of the flora in the alpine zone. *New Phytologist*, 11(2):37–50, 1912.
- [29] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.
- [30] Minoru Kanehisa and Susumu Goto. KEGG: Kyoto encyclopedia of genes and genomes. *Nucleic Acids Research*, 28(1):27–30, 2000.
- [31] David R. Karger and Clifford Stein. A new approach to the minimum cut problem. *Journal of the ACM*, 43(4):601–640, 1996.
- [32] Andrew J. Kennedy. Relational parametricity and units of measure. *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 442–455, 1997.
- [33] Chava Kimchi-Sarfaty, Jung Mi Oh, In-Wha Kim, et al. A “silent” polymorphism in the MDR1 gene changes substrate specificity. *Science*, 315(5811):525–528, 2007.
- [34] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.
- [35] Jack Kyte and Russell F. Doolittle. A simple method for displaying the hydropathic character of a protein. *Journal of Molecular Biology*, 157(1):105–132, 1982.
- [36] Matthew R. Lakin, Simon Youssef, Filippo Polo, Stephen Emmott, and Andrew Phillips. Visual DSD: A design and analysis tool for DNA strand displacement systems. *Bioinformatics*, 27(22):3211–3213, 2011.
- [37] Marjan Mernik, Jan Heering, and Anthony M. Sloane. When and how to develop domain-specific languages. *ACM Computing Surveys*, 37(4):316–344, 2005.
- [38] Luc Moreau, Ben Clifford, Juliana Freire, et al. The open provenance model core specification (v1.1). *Future Generation Computer Systems*, 27(6):743–756, 2011.
- [39] Guy M. Morton. A computer oriented geodetic data base and a new technique in file sequencing. *IBM Ltd. Technical Report*, 1966.

- [40] Saul B. Needleman and Christian D. Wunsch. A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3):443–453, 1970.
- [41] Marina V. Omelchenko, Michael Y. Galperin, Yuri I. Wolf, and Eugene V. Koonin. Non-homologous isofunctional enzymes: A systematic analysis of alternative solutions in enzyme evolution. *Biology Direct*, 5:31, 2010.
- [42] Brian D. Ondov, Todd J. Treangen, Páll Melsted, Adam B. Mallonee, Nicholas H. Bergman, Sergey Koren, and Adam M. Phillippy. Mash: Fast genome and metagenome distance estimation using MinHash. *Genome Biology*, 17(1):132, 2016.
- [43] Alan V. Oppenheim and Ronald W. Schaffer. *Discrete-Time Signal Processing*. Pearson, 3 edition, 2009.
- [44] Michael Pedersen and Gordon Plotkin. A language for biochemical systems: Design and formal specification. *Transactions on Computational Systems Biology*, 12:77–145, 2010.
- [45] Gordon D. Plotkin. A structural approach to operational semantics. *Journal of Logic and Algebraic Programming*, 60–61:17–139, 2004.
- [46] Joshua B. Plotkin and Grzegorz Kudla. Synonymous but not the same: The causes and consequences of codon bias. *Nature Reviews Genetics*, 12(1):32–42, 2011.
- [47] Predrag Radivojac, Wyatt T. Clark, et al. A large-scale evaluation of computational protein function prediction. *Nature Methods*, 10(3):221–227, 2013.
- [48] Joseph M. Replogle, Reuben A. Saunders, et al. Mapping information-rich genotype-phenotype landscapes with genome-scale Perturb-seq. *Cell*, 185(14):2559–2575, 2022.
- [49] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.
- [50] Zuben E. Sauna and Chava Kimchi-Sarfaty. Understanding the contribution of synonymous mutations to human disease. *Nature Reviews Genetics*, 12(10):683–691, 2011.
- [51] Alexandra M. Schnoes, Shoshana D. Brown, Igor Dodevski, and Patricia C. Babbitt. Annotation error in public databases: Misannotation of molecular function in enzyme superfamilies. *PLoS Computational Biology*, 5(12):e1000605, 2009.
- [52] Alexander Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, Berlin, 2003.
- [53] Jianbo Shi and Jitendra Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.
- [54] Joseph P. Simmons, Leif D. Nelson, and Uri Simonsohn. False-positive psychology: Undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological Science*, 22(11):1359–1366, 2011.
- [55] T. F. Smith and M. S. Waterman. Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1):195–197, 1981.
- [56] Martin Steinegger and Johannes Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017.

- [57] Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. *Journal of the ACM*, 44(4):585–591, 1997.
- [58] Burton H. Throckmorton. *Gospel Parallels: A Synopsis of the First Three Gospels*. Thomas Nelson, 1949.
- [59] George L. Turin. *An Introduction to Matched Filters*. IRE Transactions on Information Theory, 1960. vol. 6, no. 3, pp. 311–329.
- [60] D. A. Turner. Elementary strong functional programming. *Functional Programming Languages in Education*, pages 1–13, 1995.
- [61] Richard F. Voss. Evolution of long-range fractal correlations and  $1/f$  noise in DNA base sequences. *Physical Review Letters*, 68(25):3805–3808, 1992.
- [62] Philip Wadler. Linear types can change the world! *Programming Concepts and Methods*, 1990.
- [63] J. D. Watson and F. H. C. Crick. Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid. *Nature*, 171(4356):737–738, 1953.
- [64] Mark D. Wilkinson, Michel Dumontier, et al. The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, 3:160018, 2016.
- [65] A. A. Zamyatnin. Protein volume in solution. *Progress in Biophysics and Molecular Biology*, 24:107–123, 1972.